

Backend Interview - 50 Questions

From Zero to FAANG

Backend 50 Qs - Complete Roadmap

Section A - REST API (1-10)

1. What is REST?
2. HTTP Methods & Idempotency
3. Status Codes
4. Best Practices
5. Versioning
6. Pagination Filtering Sorting
7. HATEOAS
8. JWT Authentication
9. Rate Limiting
10. Swagger OpenAPI

Section B - Microservices (11-25)

11. Monolith vs Microservices
12. API Gateway
13. Service Discovery
14. Sync vs Async
15. Circuit Breaker
16. Retry Bulkhead Timeout
17. Saga Pattern
18. DB per Service
19. CQRS
20. Event Sourcing
21. CAP Theorem
22. Eventual Consistency
23. Observability ELK
24. Metrics Prometheus
25. Tracing Zipkin

Section C - Spring & Core (26-50)

26. Spring vs Spring Boot
27. Dependency Injection
28. Autoconfiguration
29. Core Annotations
30. Bean Lifecycle
31. Transactional
32. JPA vs Hibernate
33. N+1 Problem
34. Caching Redis
35. Kafka vs RabbitMQ
36. SQL vs NoSQL
37. Indexing
38. ACID
39. Isolation Levels
40. Sharding Replication
41. Load Balancer
42. Docker
43. Kubernetes
44. CI/CD
45. URL Shortener Design
46. Rate Limiter Design
47. LLD BookMyShow
48. SOLID Principles
49. OOP for Backend
50. Mock Interview

Save This Roadmap - 50 Qs Series Starting

Backend Interview - 50 Questions

From Zero to FAANG

Q1-Q2. What is REST API? REST Principles?

Q1. What is REST API?

Ans: REST = Representational State Transfer.
Architectural style for web services.

Uses HTTP to perform CRUD on resources.
Stateless, client-server, JSON based.

Example: GET /users/1 → get user,
POST /users → create

Key features:

- Resource based (nouns)
- Uses HTTP methods
- Stateless — each req independent
- JSON/XML response
- Cacheable.

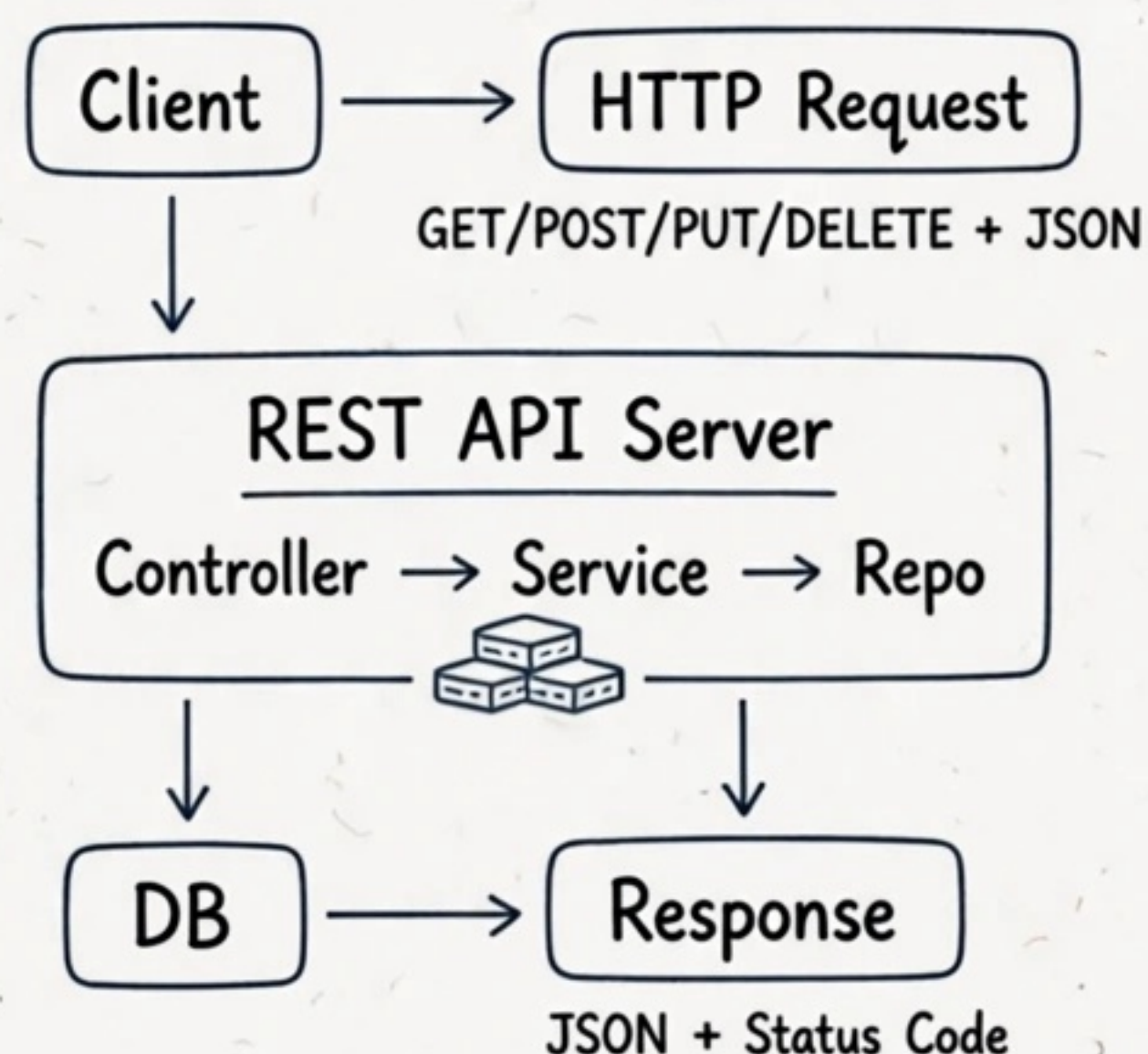
Q2. 6 Principles of REST?

1. Client-Server: separation of concerns
2. Stateless: server stores no session, each req has auth token
3. Cacheable: responses can be cached
4. Uniform Interface: same URL structure /users/{id}
5. Layered System: can have LB, Gateway layers
6. Code on Demand (optional): server can send executable code

// REST Endpoints - Resource based

GET /api/users → 200 OK List
GET /api/users/{id} → 200 Get one
POST /api/users → 201 Created
PUT /api/users/{id} → Full Update
PATCH /api/users/{id} → Partial Update
DELETE /api/users/{id} → 204 No Content

Diagram: Flow



Stateless vs Stateful

Stateless +

Request has Token
+ Data

server: no memory
across requests

Stateful +

Session stored in
server memory

server keeps
session/memory

Good vs Bad

✓ Good /users/1/orders ✓

✗ Bad /getUserOrders?id=1 ✗

Interview Tip

Tip: Say REST is resource based not action based. Always use nouns /users not verbs /getUsers. Stateless = scalable, use JWT in header.

Swipe Left → Q3-Q4: HTTP Methods & Status Codes

Part 3

Backend Interview - 50 Questions

From Zero to FAANGQ3-Q4. HTTP Methods & Status Codes?Q3. HTTP Methods CRUD?

- GET: Retrieve - safe, idempotent, cacheable
- POST: Create - not idempotent, not safe
- PUT: Full update - idempotent, not safe
- PATCH: Partial update - idempotent, not safe
- DELETE: Remove - idempotent, not safe

* Note: Idempotent = same result multiple calls

Q4. Status Codes Interview?

- 2xx Success: 200 OK, 201 Created, 204 No Content
- 3xx Redirect: 301 Moved Permanently, 304 Not Modified
- 4xx Client Error: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
- 5xx Server Error: 500 Internal Error, 502 Bad Gateway, 503 Unavailable, 504 Timeout

// Idempotency

GET /users/1 -> same result

POST /users -> creates new id each time

PUT /users/1 {name: "A"} -> same result repeated

Diagram: Methods Table

Method	Safe?	Idempotent?
GET	Yes	Yes → green
POST	No	No → red
PUT	No	Yes → blue
PATCH	No	Yes → orange
DELETE	No	Yes → purple

Diagram: Status Family (traffic light)Diagram Flow:

Invalid JSON → 400

No token → 401

No role → 403

User not found → 404

Interview Tip

Tip - GET is safe + idempotent.

POST not idempotent.

PUT/PATCH/DELETE idempotent.

Use 201 for create, 204 for delete,

400 validation, 401 no auth,

403 no permission.

Swipe Left → Q5-Q6: REST Best Practices + Versioning

Backend Interview - 50 Questions

From Zero to FAANG

Q5-Q6. REST Best Practices + Versioning?

Q5. Best Practices? ✓

- Use nouns not verbs → /users, plural nouns
- Use methods correctly: GET • POST • PUT • PATCH • DELETE
- Proper status codes: 200 • 201 • 204 • 400 • 404
- Versioning: /api/v1
- Pagination: ?page=1&limit=10
- Filtering: ?role=admin
- HATEOAS: include _links for related resources
- Consistent error JSON response
- JWT for auth • HTTPS always • Rate limit requests

✓ Q6. Versioning types?

- URI → /api/v1/users (common)
- Query → ?version=1
- Header → Accept: application/vnd.codeera.v1+json
- Content Negotiation

→ URI is simplest & most used ↩

✓ Best Practice ✓ vs ✗

✓ Good: /users/{id}

+ Bad: /getUser?id=1

Pagination:

[page: 1] → [page: 2] → [page: 3]

limit = 10 per page → ?page=1&limit=10

Versioning Comparison:

Type	Pros	Cons
URI	simple • visible • cacheable	in URL path
Header	clean • no URL change harder to debug	hidden •
Query	flexible • easy test mix • less	mix with less standard

Error JSON:

```
{
  "code": 404,
  "message": "User not found",
  "details": "User id 123 does not exist"
}
```

Interview Tip: ★ ★
Keep API predictable & consistent.
Version early, document clearly.

Swipe Left → Q7-Q8: Monolith vs Microservices

Part 5

@codeera.tech

Backend Interview - 50 Questions

From Zero to FAANG

Q7-Q8. Monolith vs Microservices?

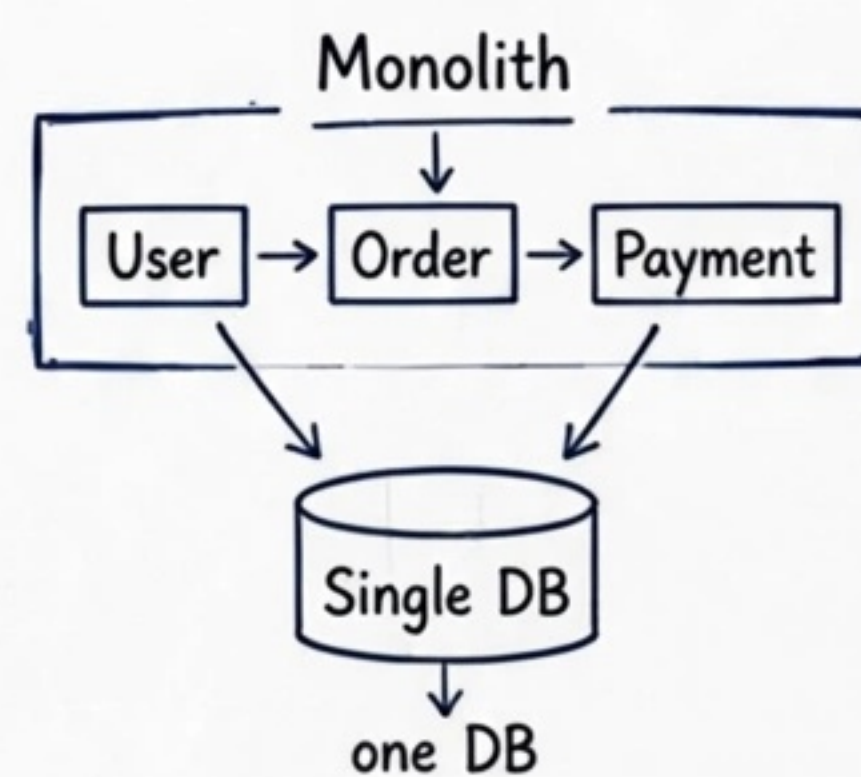
Q7. Monolith vs Micro?

- Monolith: single codebase • single DB • single deploy • tight coupling • hard to scale • failure affects all.
- Microservices: small independent services • own DB • own deploy • loose coupling • scalable • isolated failure • polyglot • complex.

Q8. Challenges?

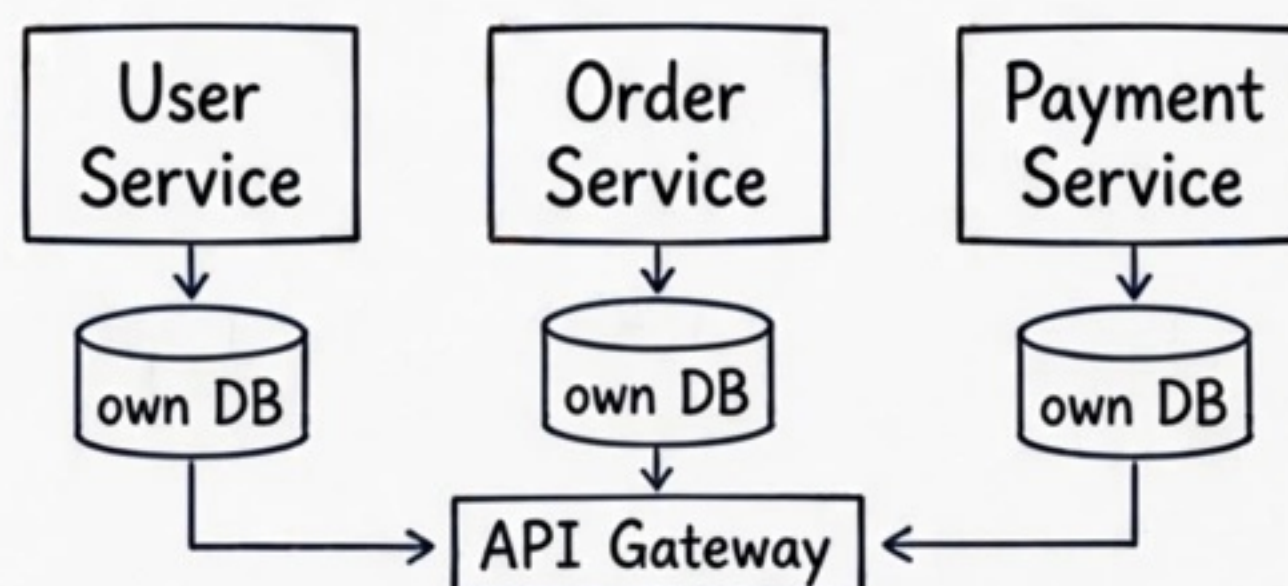
- Distributed transactions → Saga
- Service discovery
- Config management
- Latency
- Eventual consistency
- Monitoring: ELK, Zipkin
- K8s complexity
- Network failure

Diagrams:



VS

Microservices



Message Queue: → Kafka

Comparison Table

Feature	Monolith	Microservices
Scalability	Low	High
Deployment	Single	Independent
Fault Tolerance	Low	High

Swipe Left → Q9-Q10: API Gateway + Service Discovery

Part 6

Backend Interview - 50 Questions

From Zero to FAANG

Q9-Q10. API Gateway + Service Discovery?Q9. What is API Gateway?

Ans: Single entry point for all clients.

Responsibilities:

- Routing, Authentication JWT, Rate Limiting, Load Balancing,
- Request Aggregation, Caching, Logging, SSL termination.

Examples: Kong, Netflix Zuul, Spring Cloud Gateway, AWS API Gateway.

Q10. What is Service Discovery?

Why Need? Dynamic IPs in containers

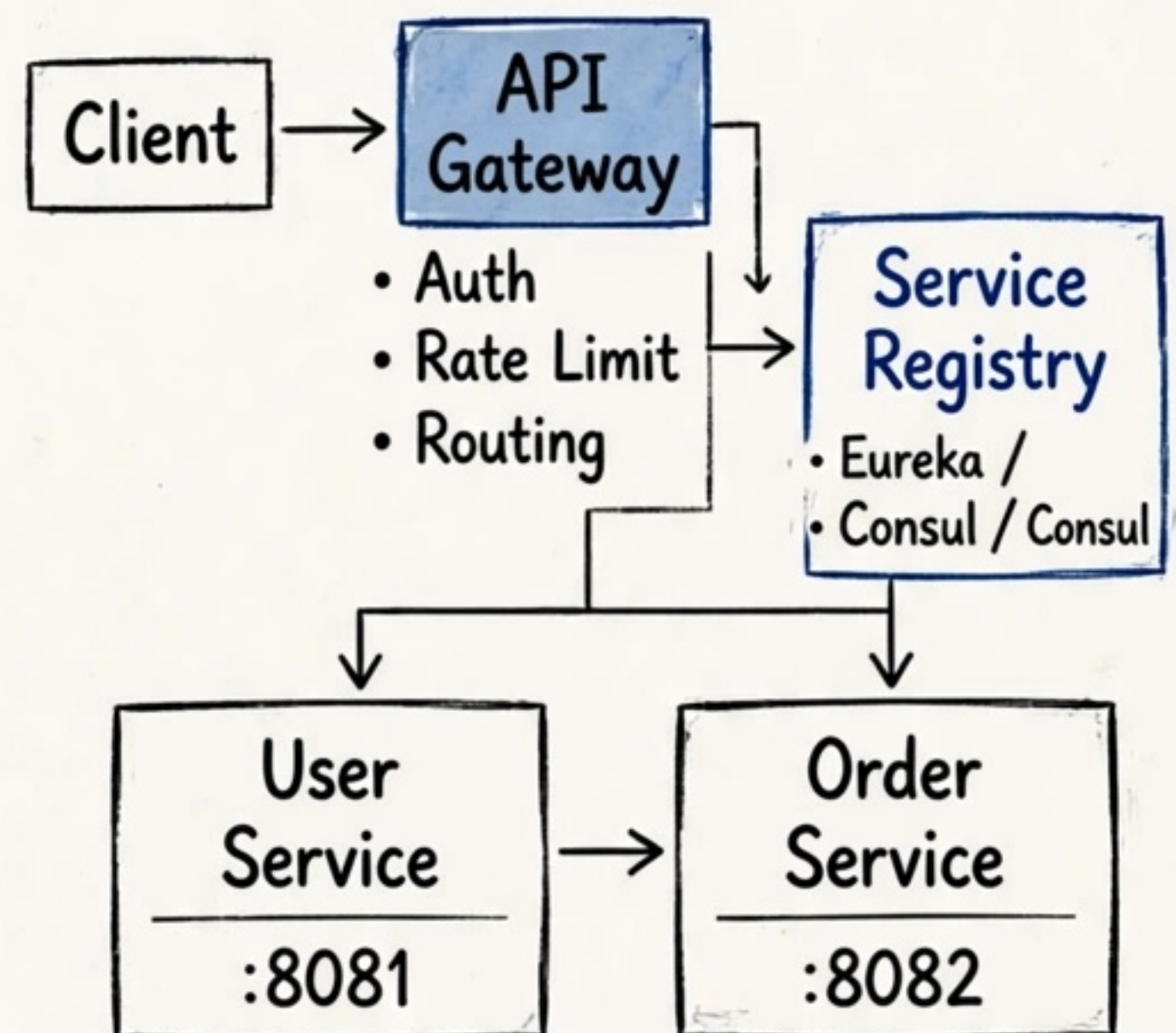
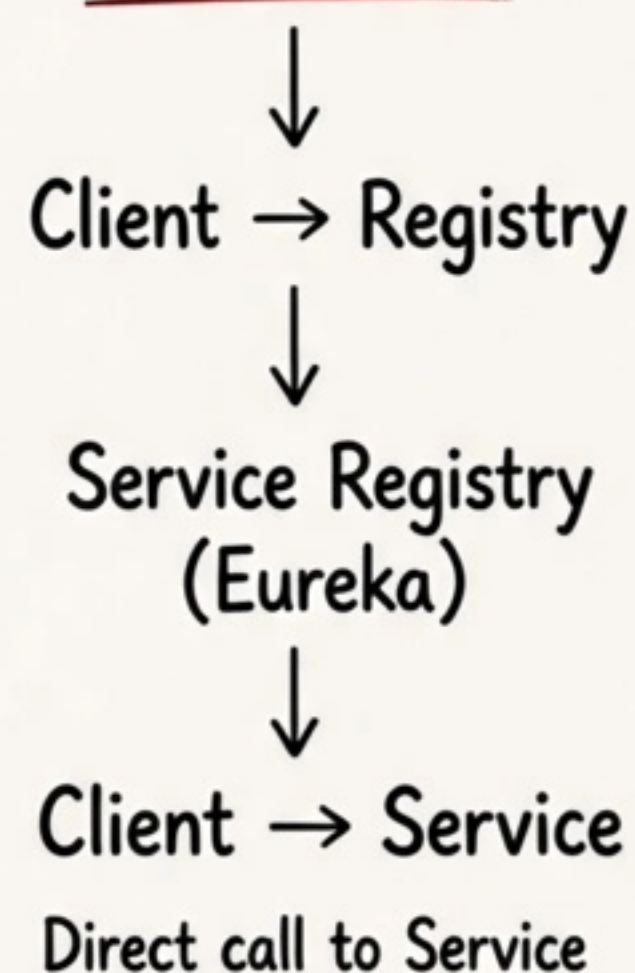
- K8s • Auto-scaling.
- Need Registry.

Two Types:

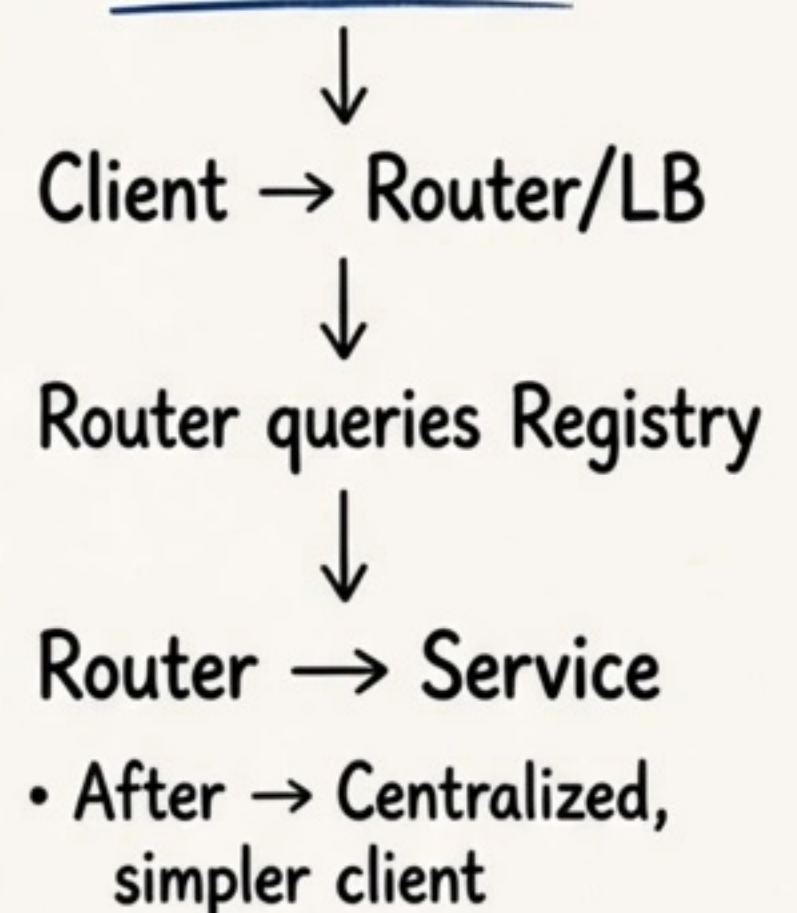
- Client-side: Client queries Service Registry (Eureka) then calls service directly.
- Server-side: Client calls Router/LB, Router queries Registry (Consul, K8s DNS, ELB) then forwards.

Tools:

- Eureka, Consul, etcd, Zookeeper, K8s Service Discovery.

System Flow:Comparison:Client-side

Before → Faster, Client logic

Server-side

Registry Tools:

- Consul • etcd
- Zookeeper
- K8s DNS

Swipe Left → Q11-Q12: Sync vs Async Communication →

Part 7

Backend Interview - 50 Questions

From Zero to FAANG

Q11-Q12. Sync vs Async Communication?Q11. Sync (Synchronous)

- Request waits → blocking
- Simple, immediate, easy
- Tight coupling
- Cascade failure • Latency

Use for: get user details

```
restTemplate.getForObject(...)-
→ waits for response → returns result
```

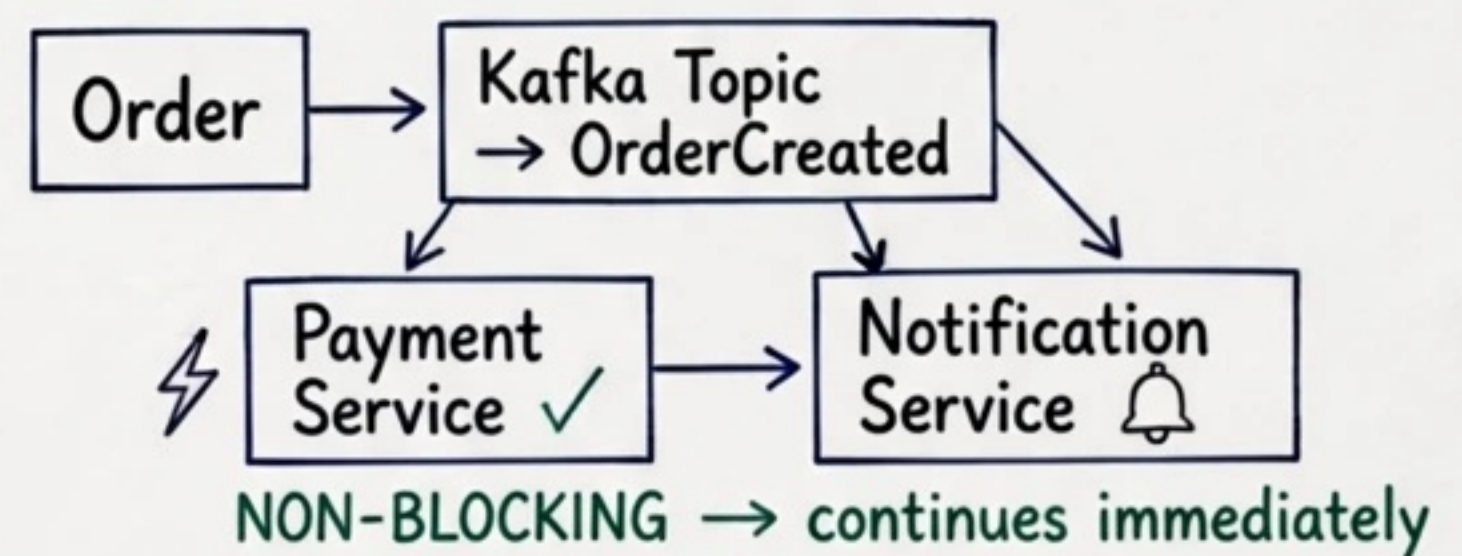
Q12. Async (Asynchronous)

- Non-blocking • pub/sub
- Loose coupling • resilient • scalable
- Buffer → eventual consistency
- Complex debugging • need idempotency
- Event-driven

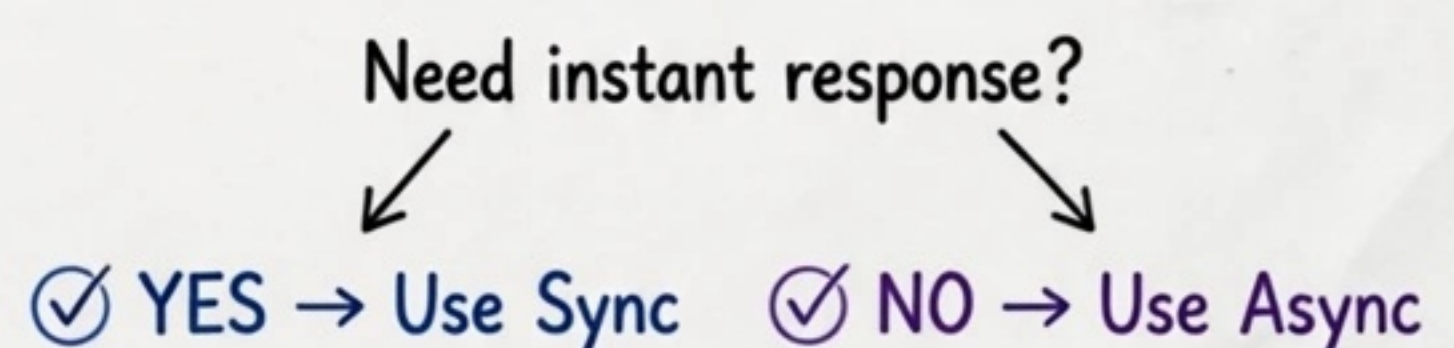
Use for: email • analytics • order → payment • notification

```
kafkaTemplate.send(event)
→ publishes event to topic
```

consumer/listener → processes later

Diagrams & ComparisonSync FlowAsync FlowComparison Table

Metric	Sync	Async
1. Latency	Higher (waits)	Lower (no wait)
2. Coupling	Tight	Loose
3. Failure Risk	Cascade failure	Isolated / Resilient
4. Consistency	Immediate	Eventual

Decision Tree

Swipe Left → Q13-Q14: Circuit Breaker + Resilience

Part 8

Backend Interview - 50 Questions

From Zero to FAANG

Q13-Q14. Circuit Breaker + Resilience?

Q13. Circuit Breaker —
prevent cascade failure

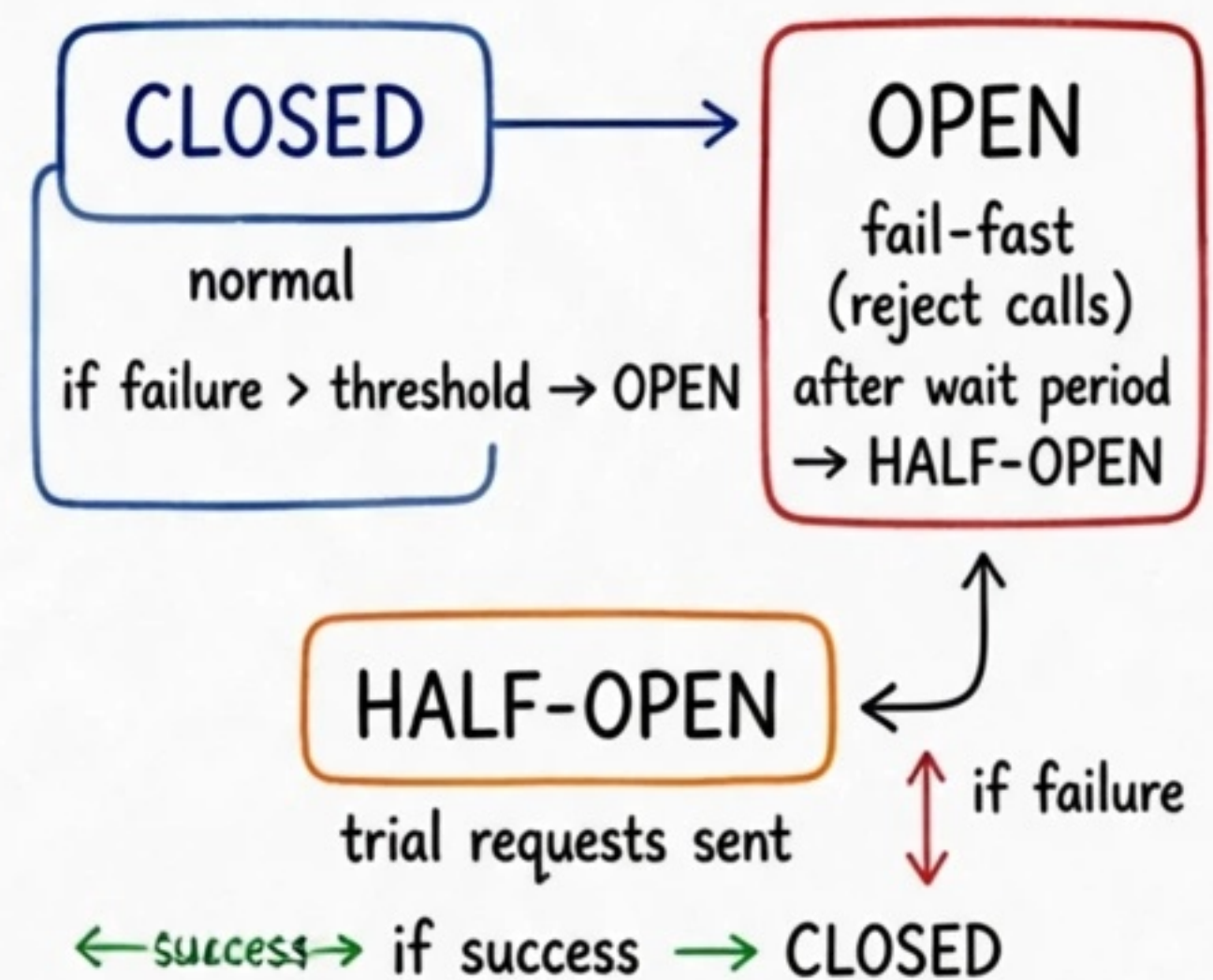
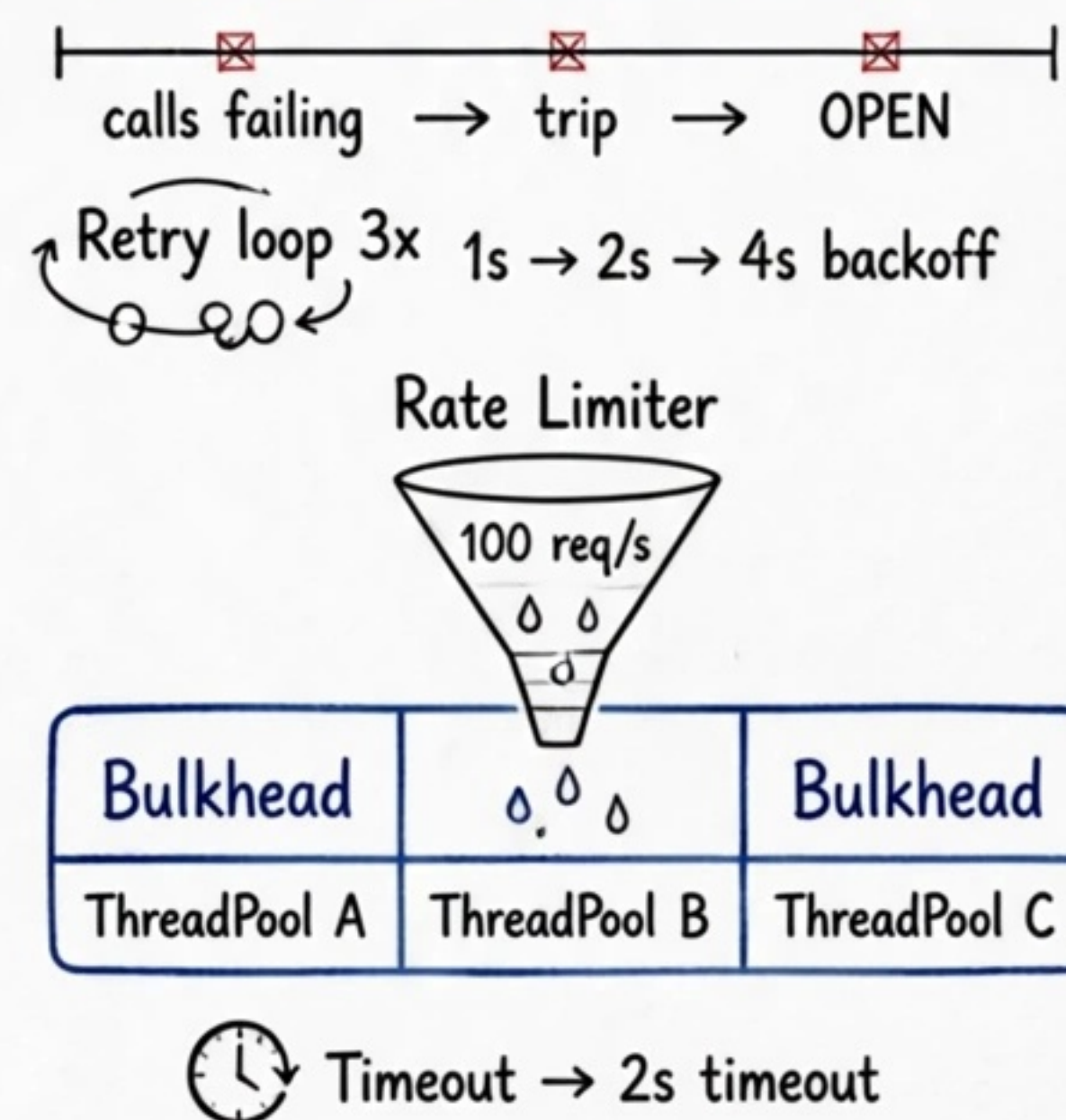
- States:
 - CLOSED → normal operation, requests pass
 - OPEN → fail-fast, no calls to downstream
 - HALF-OPEN → test if recovered (trial calls)
- Tools:
 - Resilience4j (modern, recommended)
 - Hystrix (deproceted)
 - fallback for graceful response

Q14. Other Resilience Patterns:

- Retry → 3x attempts with exponential backoff
- Timeout → avoid blocking calls, fail fast
- Bulkhead → isolate thread pools (limit concurrency)
- Rate Limiter → 100 req/sec to protect service
- Fallback → default response / cached result

Code:

```
@CircuitBreaker(name="userService",
fallbackMethod="fallback")
@Retry(name="userService")
@RateLimiter(name="userService")
```

Circuit Breaker State DiagramTimeline & Patterns

Swipe Left → Q15-Q16: DB per Service + Saga

Backend Interview - 50 Questions

From Zero to FAANG

@codeera.tech

Part 9

Q15-Q16. DB per Service + Saga Pattern?

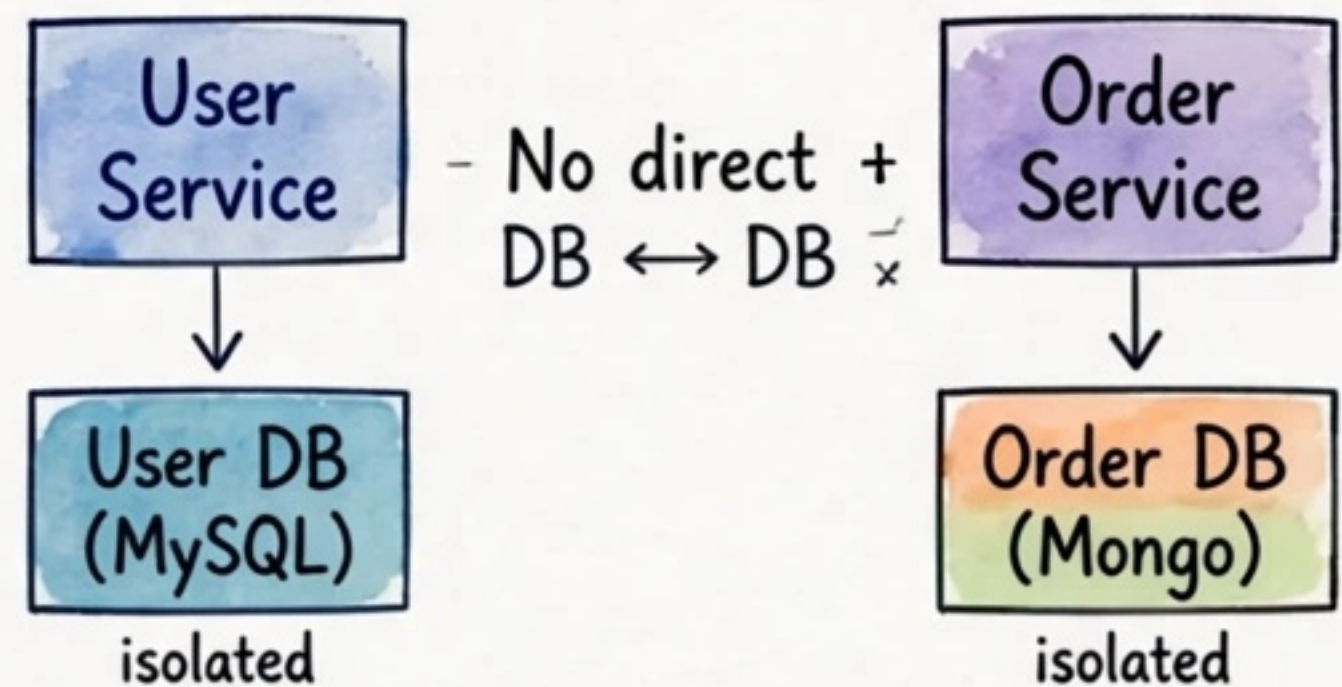
Q15. DB per Service?

- Each service owns its DB, cannot access other DB directly.
- access via API only.
- Benefits: loose coupling, independent scaling polyglot persistence (User → MySQL, Order → Mongo)
- Challenge: joins difficult → via API composition / CQRS.
- Anti-pattern: Shared DB.

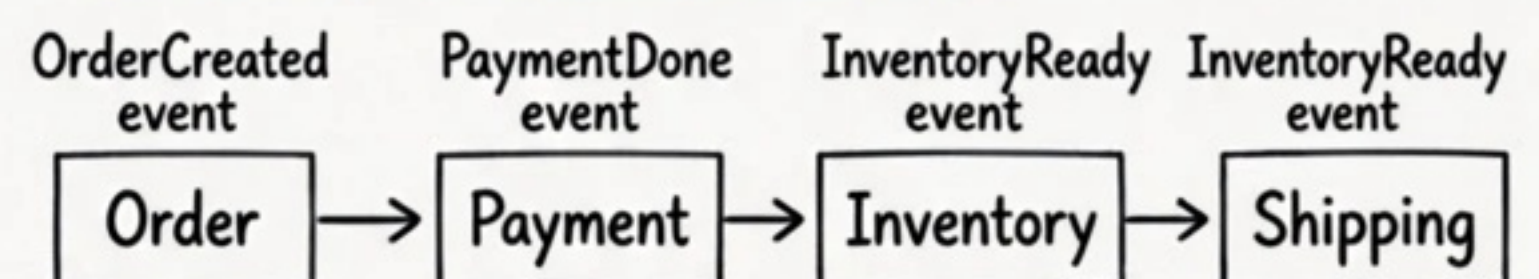
Q16. Saga for distributed transactions?

- For microservices transactions across DBs.
- 2 types:
 - Choreography: each service publishes event, next listens, no central coord
 - Orchestration: central orchestrator Coordinates flow
- Example: Order PENDING → OrderCreated event → Payment pays → PaymentDone → Inventory → Shipping
- If fails → compensating transactions: refund, CANCELLED.
- Tools: Camunda, Eventuate.

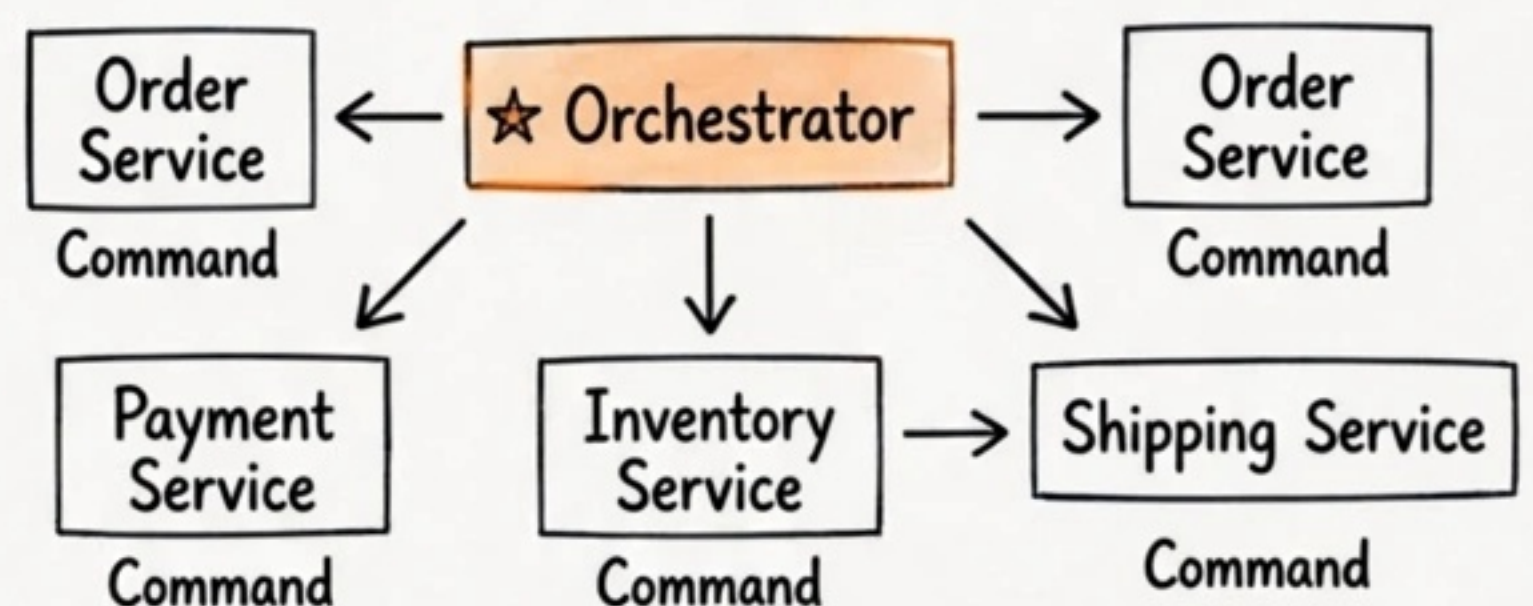
DB per Service



Saga — Choreography



Saga — Orchestration



Compensating / Rollback → + refund, cancel

Choreo vs Orchestrator

Choreography		Orchestrator
Decentralized	✓	× Centralized
Complexity	Simple	Complex

Swipe Left → Q17-Q18: CQRS + Event Sourcing

Part 10

Backend Interview - 50 Questions

From Zero to FAANG

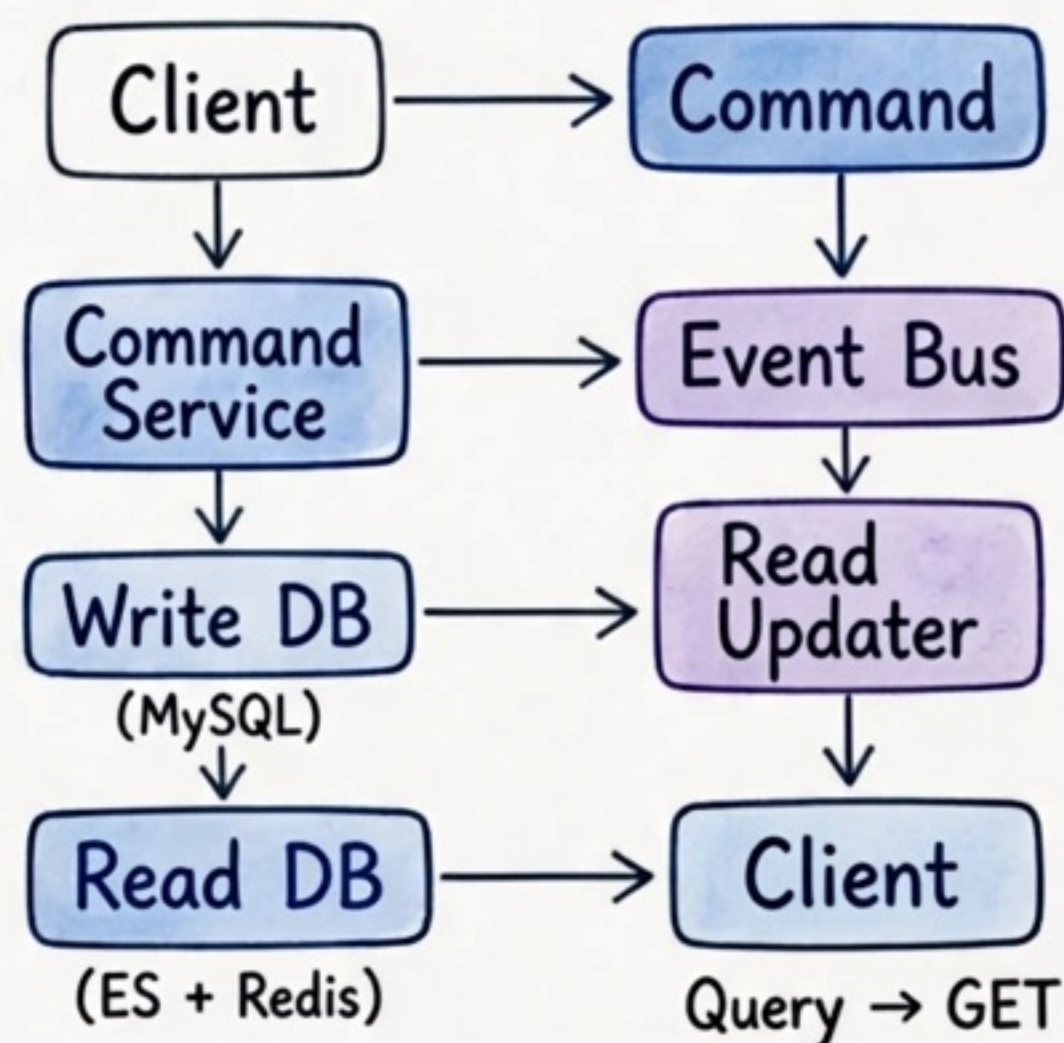
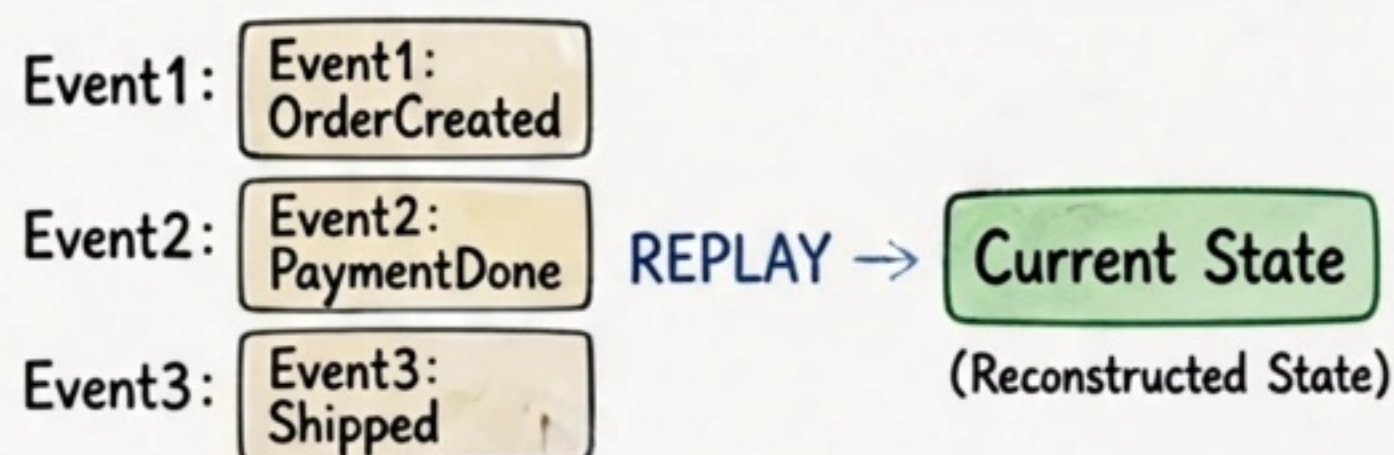
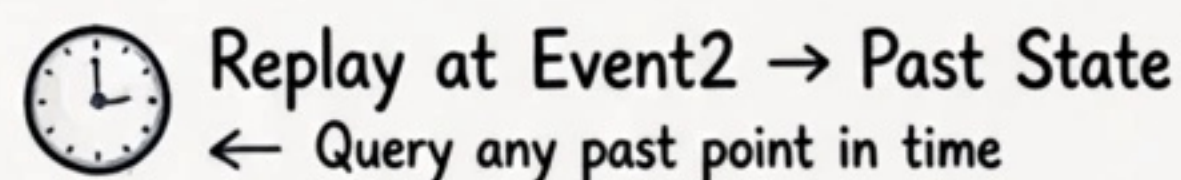
Q17-Q18. CQRS + Event Sourcing?Q17. CQRS?

Command Query Responsibility Segregation
— separate read & write models

- Command: write → POST / PUT / DELETE → Write DB
- Query: read → GET → Read DB (replicated / cache)
- Benefits:
 - scale independently
 - optimized schemas
 - faster reads
- Write: MySQL — normalized
- Read: ES / Elasticcharch — denormalized + Redis

Q18. Event Sourcing?

- Store sequence of events, not current state
- Event Store = immutable log → to get state, replay events
- Benefits:
 - audit trail • time travel
 - rebuild state
 - debugging
- Example: OrderCreated → PaymentDone → Shipped → Delivered
- Tools: Kafka, Axon
- Code Flow: CommandService save → Write DB → publishes "UserCreated" → QueryService updates Read DB / ES, replay events

1. CQRS Architecture Flow2. Event Store Log → Replay3. Time Travel4. Comparison: Traditional vs Event Sourcing

Traditional	Event Sourcing
Stores current state	Stores event history
Update overwrites data	Append-only immutable log
Hard to audit/debug	Easy audit & replay

Swipe Left → Q19-Q20: CAP + Eventual Consistency

Backend Interview - 50 Questions

From Zero to FAANG

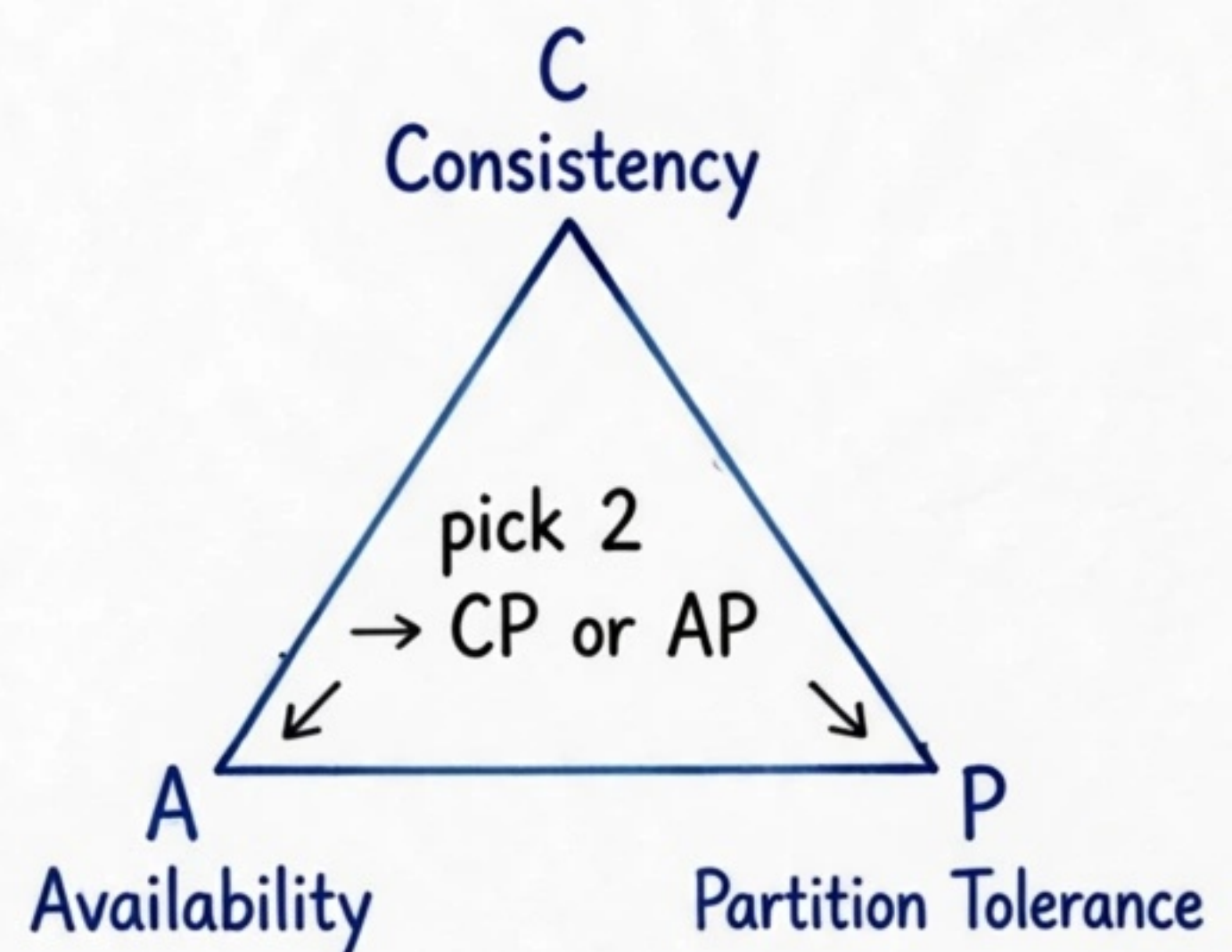
Q19-Q20. CAP Theorem + Eventual Consistency?

Q19. CAP Theorem?

In distributed system you can only have 2 of 3:

- **Consistency** (all nodes same data)
- **Availability** (every request gets response)
- **Partition Tolerance** (system works despite network partition)

Real world → **P is mandatory**, choose **CP** (**Consistency + Partition**) e.g. **MongoDB • HBase** vs **AP** (**Availability + Partition**) e.g. **Cassandra • Dynamo**



CP →	AP →
MongoDB • HBase (Consistent)	Cassandra • DynamoDB (Available)

Q20. Eventual Consistency?

Not immediate but eventually all replicas consistent.

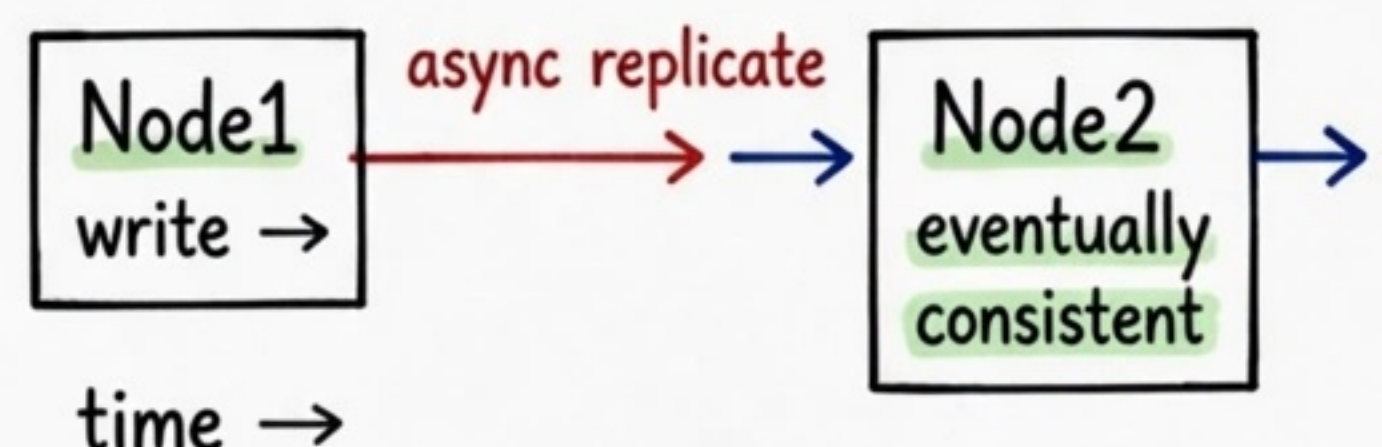
BASE vs **ACID**. **BASE**: Basically Available
• Soft state • Eventual consistent.

Used in **AP** systems → Kafka • Cassandra.

How to handle? / **vector clocks**

- **Read repair**
- **Anti-entropy**

Eventual consistency timeline:



Swipe Left → Q21-Q22: Eventual Consistency + Observability

Backend Interview - 50 Questions

From Zero to FAANG

Q21-Q22. Eventual Consistency vs Strong + Observability?

Q21. Strong vs Eventual?

Ans: Strong - immediate, all nodes same data. High latency, CP system.
Example: Banking txn needs strong consistency.

Eventual - low latency, AP system, eventually same across nodes.
Example: Social feed, likes eventual OK.

Strong vs Eventual — Comparison

Aspect	Strong	Eventual
Latency	High	Low
Consistency	Immediate	Eventual
Type	CP	AP
Use case	Banking	Social Feed

Q22. Observability 3 pillars?

Ans:

1. Logs - ELK Stack: Elasticsearch, Logstash, Kibana. Centralized logs searchable.
2. Metrics - Prometheus + Grafana. CPU, Memory, Latency, QPS, Error rate. Alerts.
3. Tracing - Distributed tracing Zipkin / Jaeger. traceId spanId across services.

Extra: Health checks /actuator/health, Centralized config Spring Cloud Config, Consul, Vault.

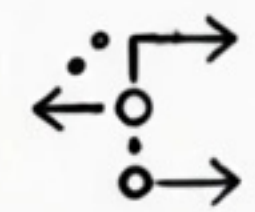
Observability pillars:



Logs

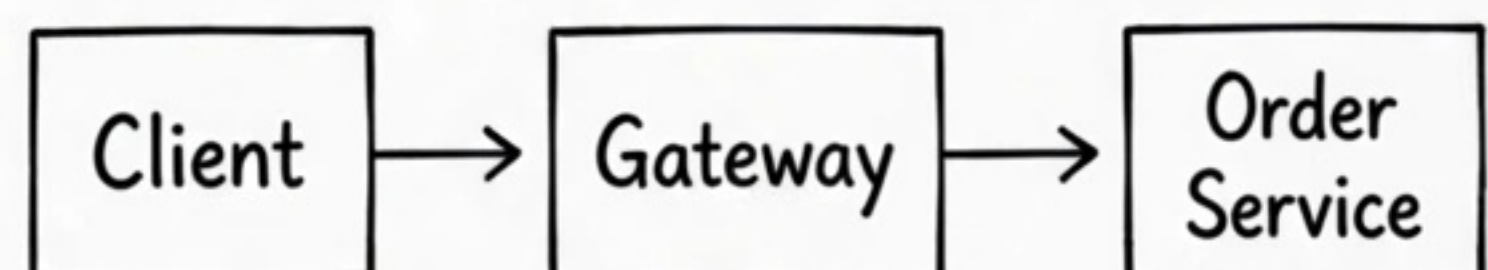


Metrics



Tracing

Tracing flow diagram:



traceId=abc-123 propagates across all services

Swipe Left -> Q23-Q24: Metrics + Tracing

Part 13

Backend Interview - 50 Questions From Zero to FAANG

@codeera.tech

Q23-Q24. Metrics Prometheus + Tracing Zipkin?

Q23. Metrics with Prometheus?

Ans: Pull based metrics, app exposes /metrics endpoint.

Prometheus scrapes periodically, PromQL query language.

Grafana dashboards + alerts (CPU >80%, latency p99).

Key metrics: RED = Rate • Errors • Duration,
• USE = Utilization • Saturation • Errors.

Q24. Distributed Tracing Zipkin?

Ans: Need to trace request across microservices.

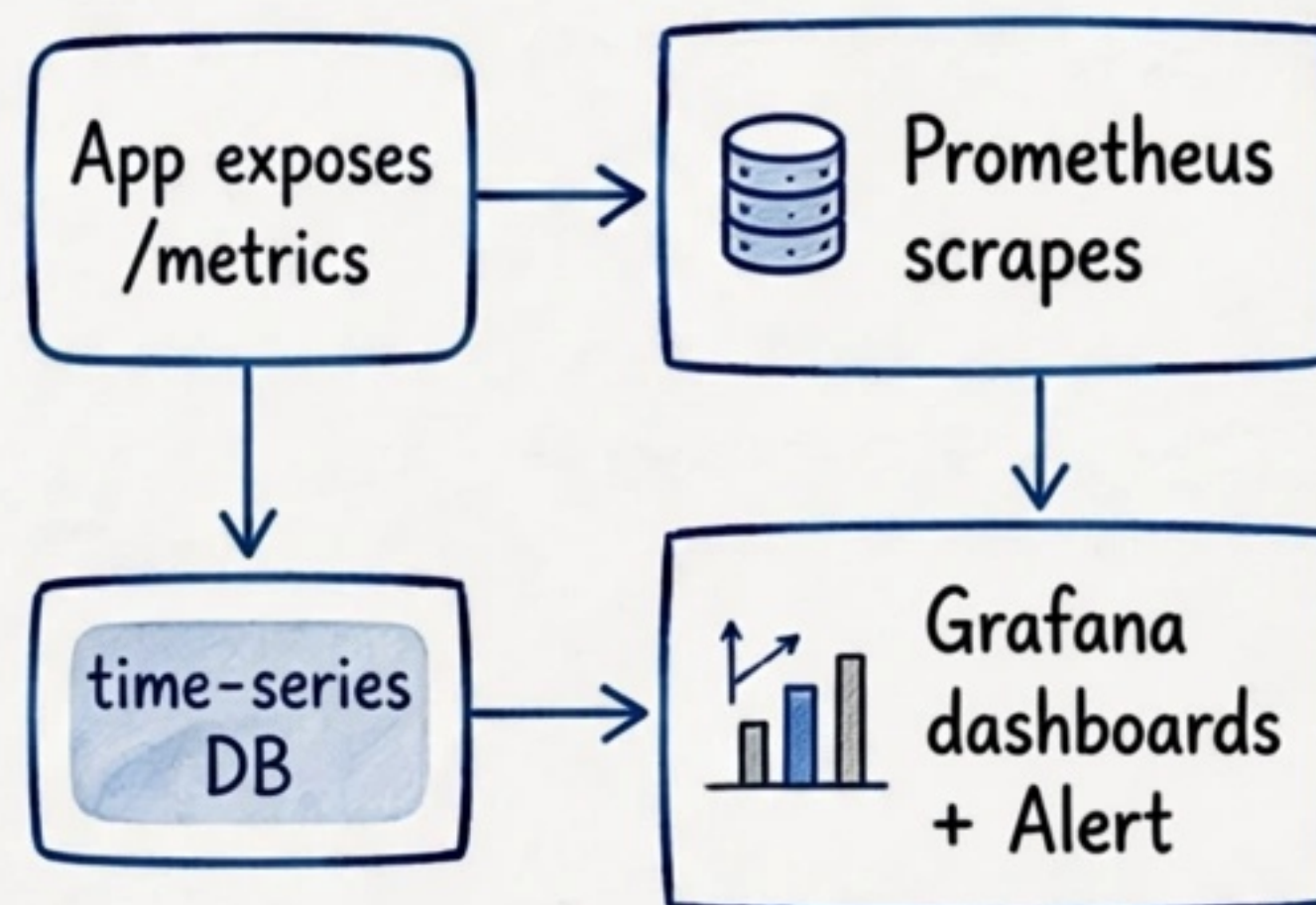
Each request = traceId, each hop = spanId, parent span link.

Zipkin collects spans, shows waterfall timeline, latency, bottleneck.

Example: traceId 123 -> Gateway 10ms -> User Service 30ms -> Order DB 20ms = total 60ms, identify slow service.

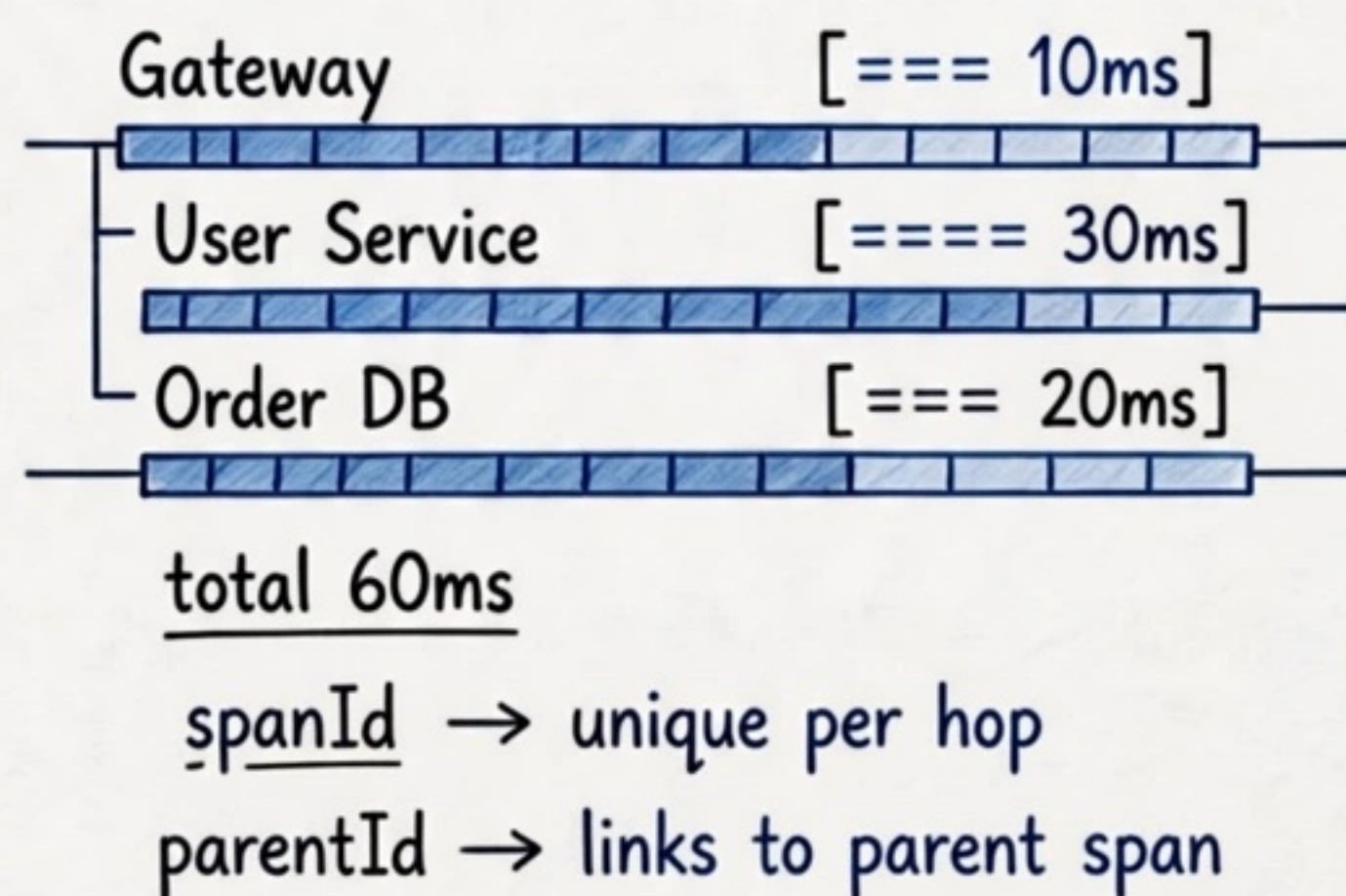
Tools: Jaeger, Zipkin, Sleuth + Zipkin.

Prometheus architecture flow:



Zipkin Trace Timeline (Waterfall)

TraceId: 123



Swipe Left -> Q25-Q26: Security + Spring Boot Start

Backend Interview - 50 Questions

From Zero to FAANG

Q25-Q26. Security in Microservices + Spring vs Boot?

Q25. Security in Microservices?

Ans:

- API Gateway - JWT validation + OAuth2
- mTLS service-to-service mutual TLS certs
- Rate limiting + throttling
- No direct DB access - via API only
- Secrets in Vault / AWS Secrets Manager
- HTTPS everywhere TLS 1.2+
- Validate input → OWASP Top 10

Q26. Spring vs Spring Boot?

Ans:

Spring:

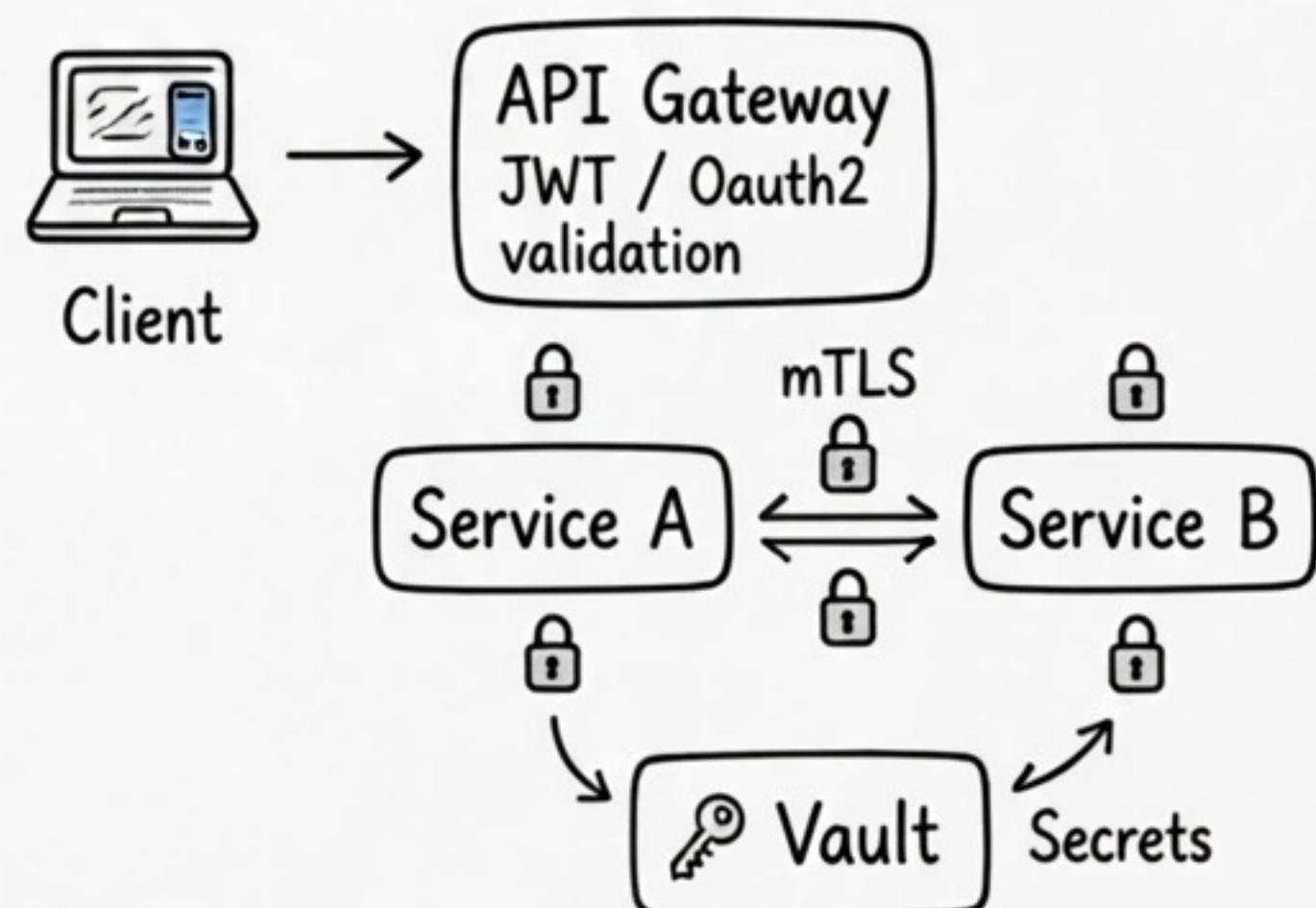
- Framework for DI, AOP, MVC
- Needs manual XML / Java config
- Heavy setup

Spring Boot:

- Auto-config + Starters
- Convention over configuration
- Embedded Tomcat/Jetty
- Production ready: Actuator, metrics

Formula: Boot = Spring + Auto-config + Starter + Actuator + Embedded server

Security Flow:



Spring vs Boot Comparison

Feature	Spring	Spring Boot
Config:	XML manual	Auto-config
Server:	External Tomcat	Embedded Tomcat
Setup:	Manual	Starter quick
Prod ready:	No	Actuator yes

Boot = Spring + Auto-config + Starter + Actuator + Embedded server

Swipe Left → Q27-Q28: Dependency Injection + AutoConfig

Part 15

Backend Interview - 50 Questions

From Zero to FAANG

Q27-Q28. DI + Autoconfiguration?

Q27. Dependency Injection?

Ans: Design pattern - object receives dependencies rather than creating them.

Types:

- Constructor (recommended) - final, testable
- Setter - optional deps
- Field (avoid) - hidden deps

Benefits: loose coupling, testable, maintainable

Spring: @Autowired injects, Container manages

Q28. Autoconfiguration?

Ans: Spring Boot magic!

@SpringBootApplication = @Configuration +
@EnableAutoConfiguration + @ComponentScan

Checks classpath:

- if JPA present → configures DataSource
- if Web present → configures Tomcat

How?

spring.factories / AutoConfiguration classes

@ConditionalOnClass, @ConditionalOnMissingBean

Can override via application.properties

DI Diagram:

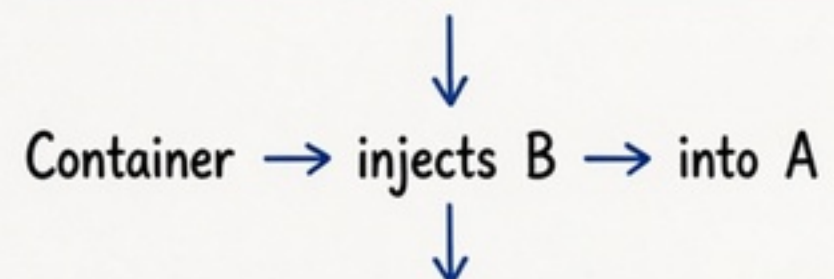
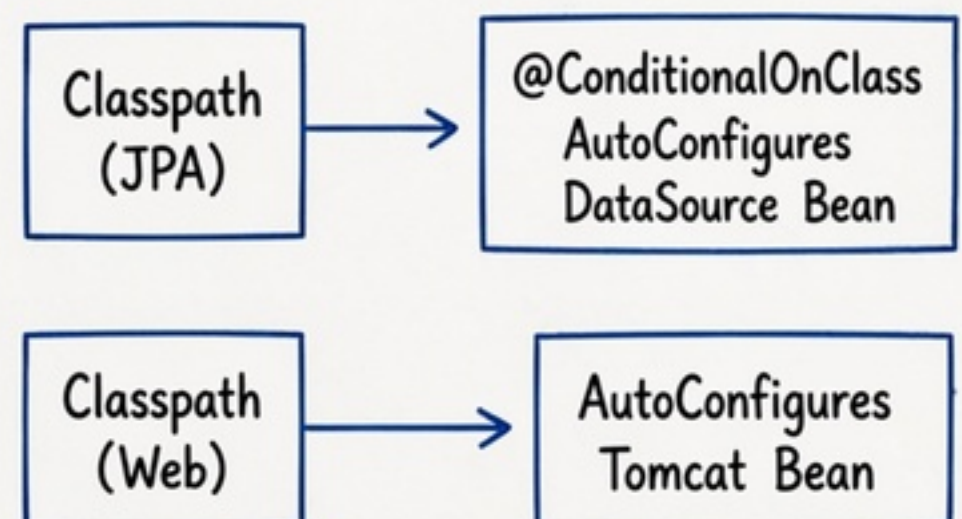
Box: Class A has dependency B →
show comparison:

LEFT bad:

new B() = ✗
tight coupling
[red X]

RIGHT good:

Container injects
B Loose coupling
✓

Autoconfiguration Flow:

* @SpringBootApplication triggers scan

Swipe Left → Q29-Q30: Core Annotations + Bean Lifecycle

Part 16

Backend Interview - 50 Questions

From Zero to FAANG

Q29-Q30. Core Annotations + Bean Lifecycle?

Q29. Core Annotations?

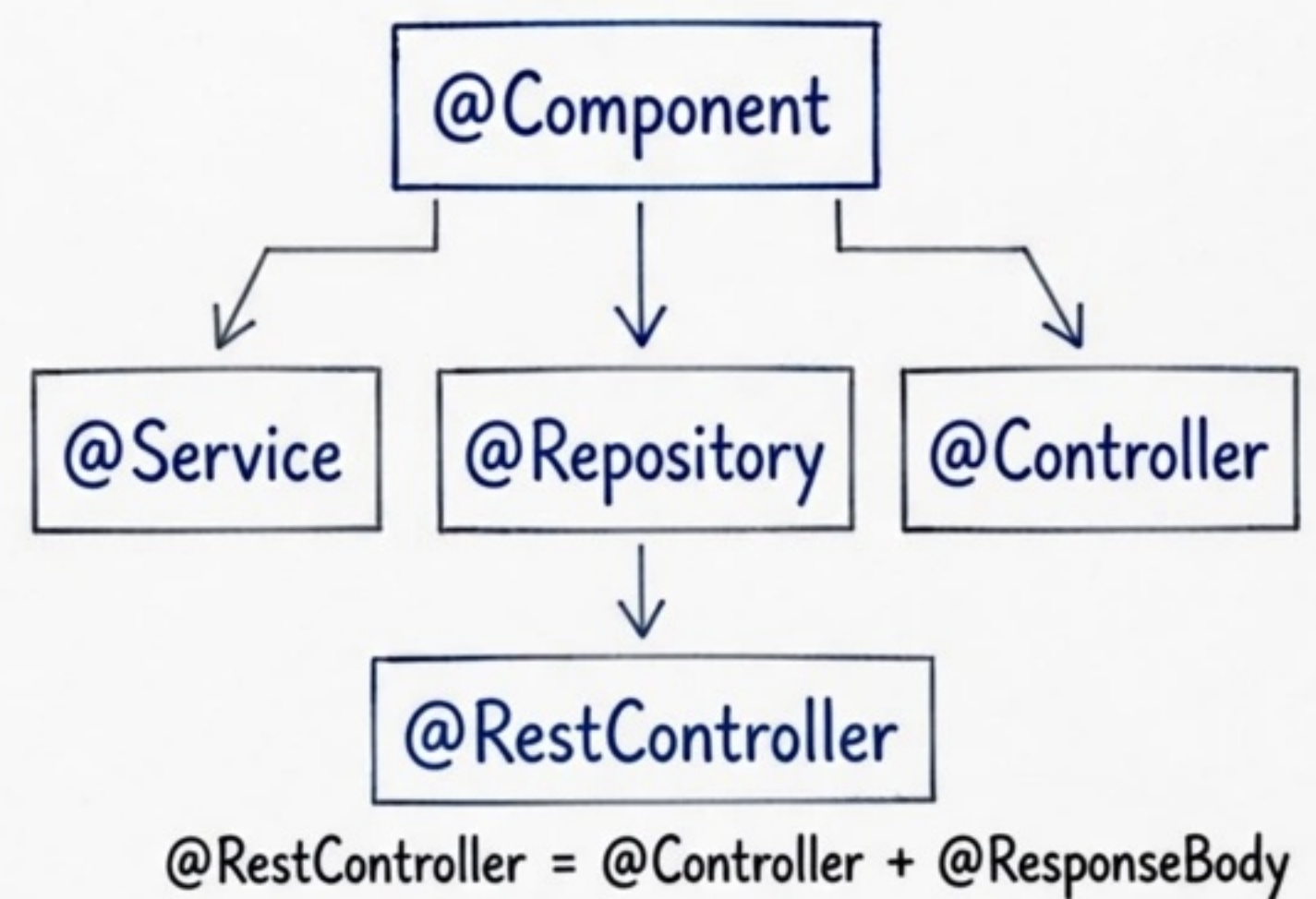
- @Component generic bean
- @Service business logic
- @Repository DAO exception translation
- @Controller MVC
- @RestController = @Controller + @ResponseBody
- @Autowired DI
- @ComponentScan scans packages
- @Configuration defines beans
- @Bean method bean
- @Value injects property.

Q30. Bean Lifecycle?

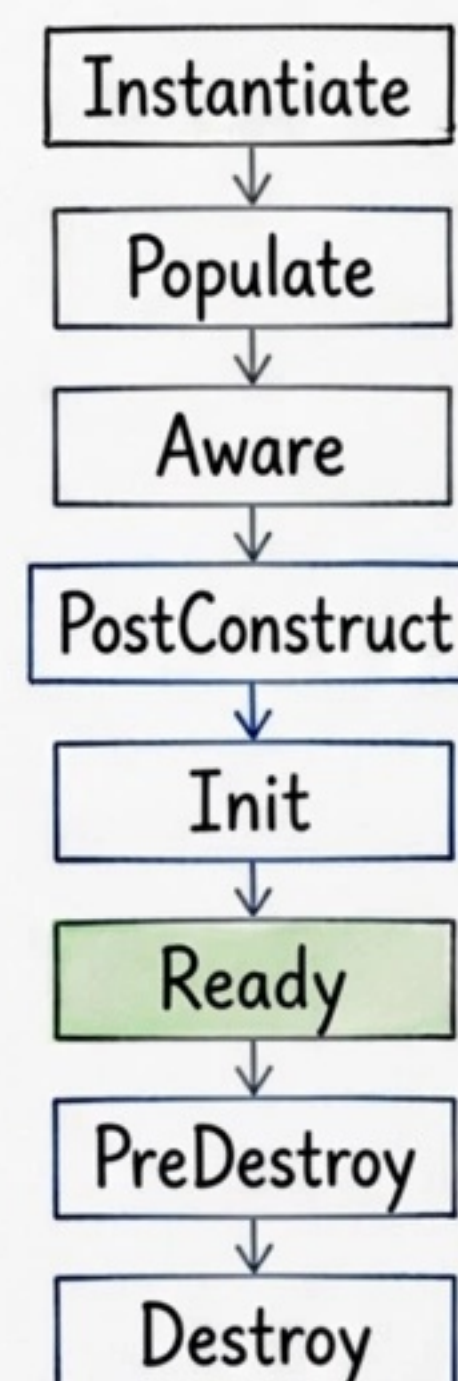
Container creates → Dependency injection →
BeanNameAware → BeanFactoryAware →
@PostConstruct → InitializingBean afterPropertiesSet →
Custom initMethod → Bean Ready → @PreDestroy →
DisposableBean destroy → close.

Scopes: singleton (default) • prototype • request • session.

Annotations Hierarchy



Bean Lifecycle timeline



Swipe Left → Q31-Q32: Transactional + JPA vs Hibernate

Part 17

Backend Interview - 50 Questions

From Zero to FAANG

Q31-Q32. @Transactional + JPA vs Hibernate?

Q31. @Transactional?

Ans: Manages transaction ACID.

Propagation:

- REQUIRED (default): join existing or new
- REQUIRES_NEW: suspend and new
- NESTED: nested savepoint
- SUPPORTS, MANDATORY, NOT_SUPPORTED, NEVER, etc.

Isolation: READ_COMMITTED is default, also READ_UNCOMMITTED, REPEATABLE_READ, SERIALIZABLE

Rollback: for RuntimeException only; not checked exceptions (use rollbackFor to include checked)

Use: @Transactional on service layer

Pitfall: self invocation proxy doesn't work!

Use AOP proxy or inject self bean

Q32. JPA vs Hibernate?

Ans: JPA is spec / interface — javax.persistence; EntityManager

Hibernate is the implementation (ORM)

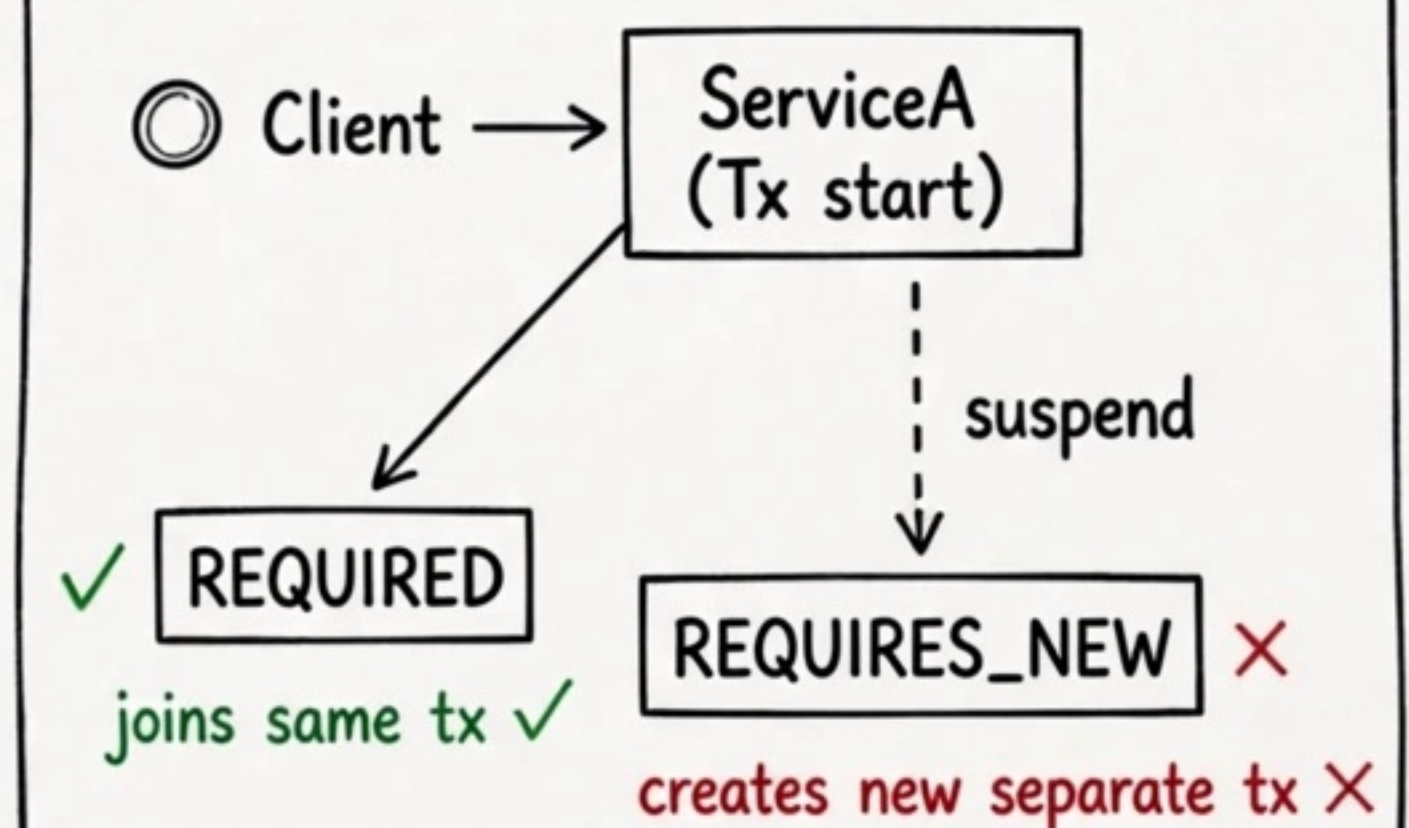
JPA defines @Entity, @Table, @Id

Hibernate implements plus extra features:

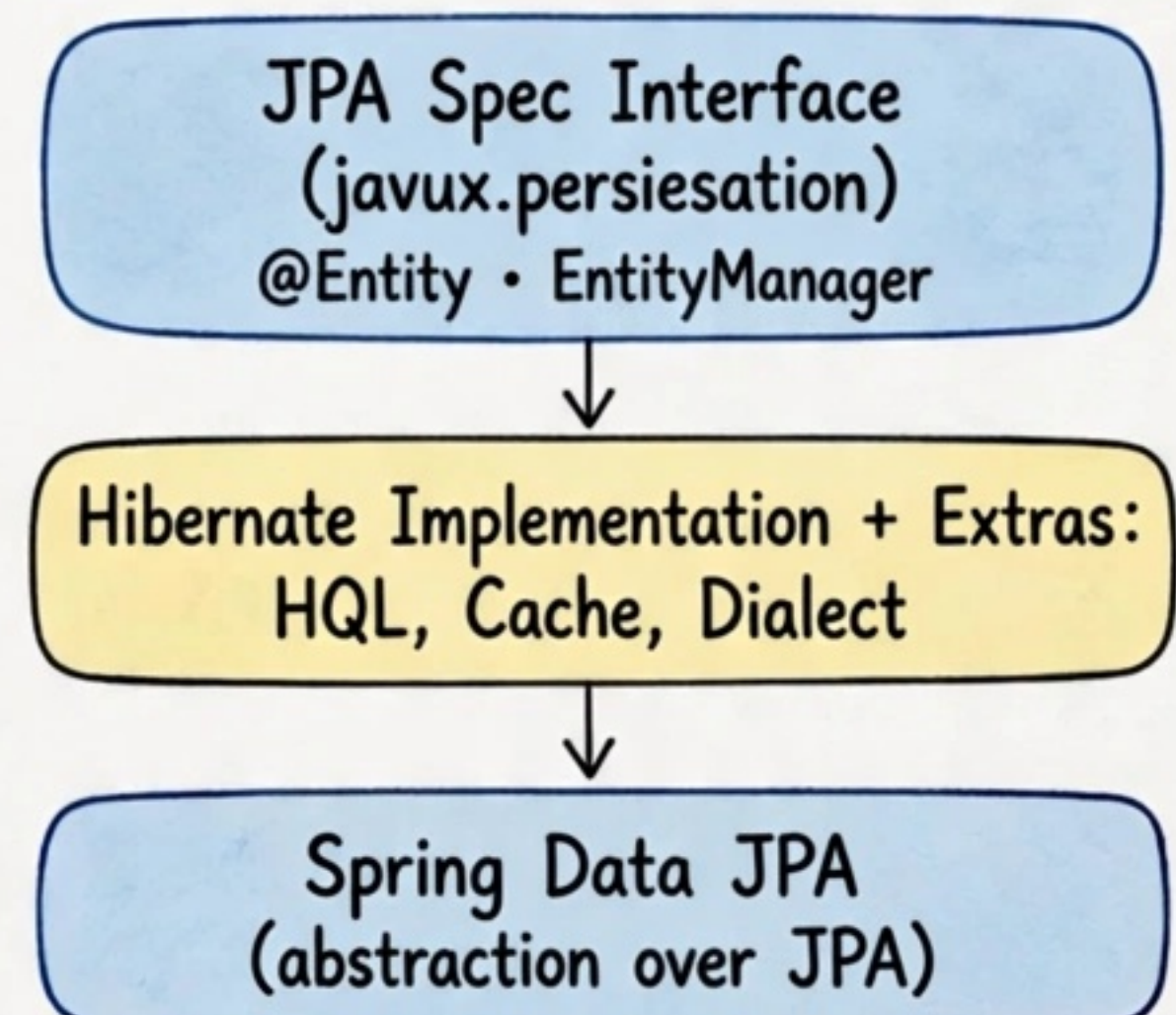
Dialect, L1/L2 cache, HQL, dirty checking

Spring Data JPA uses Hibernate underneath

Transaction Propagation:



JPA vs Hibernate Hierarchy:



Interview Tip

Tip: Always put @Transactional on the service layer, not controller. Rollback only for unchecked exceptions by default. Self-call fails due to proxy.

Swipe Left -> Q33-Q34: N+1 + Caching Redis

Part 18

Backend Interview - 50 Questions

From Zero to FAANG

Q33-Q34. N+1 Problem + Caching Redis?

Q33. N+1 Problem?

Ans: 1 query for parent + N queries for children = slow.

Example: get 100 users each with orders = 1 + 100 queries = 101 queries!
slow performance.

✓ Fix:

- JOIN FETCH, @EntityGraph, batch
- size @BatchSize, DTO projection,
- SELECT IN.

Q34. Caching? Redis?

Ans: Spring @Cacheable @CacheEvict @CachePut with Redis.

Cache aside pattern:

check cache → if miss → go DB then put cache.

TTL, eviction policy LRU • LFU • FIFO,
cache stampede prevention (mutex, warmup).

Redis data types:

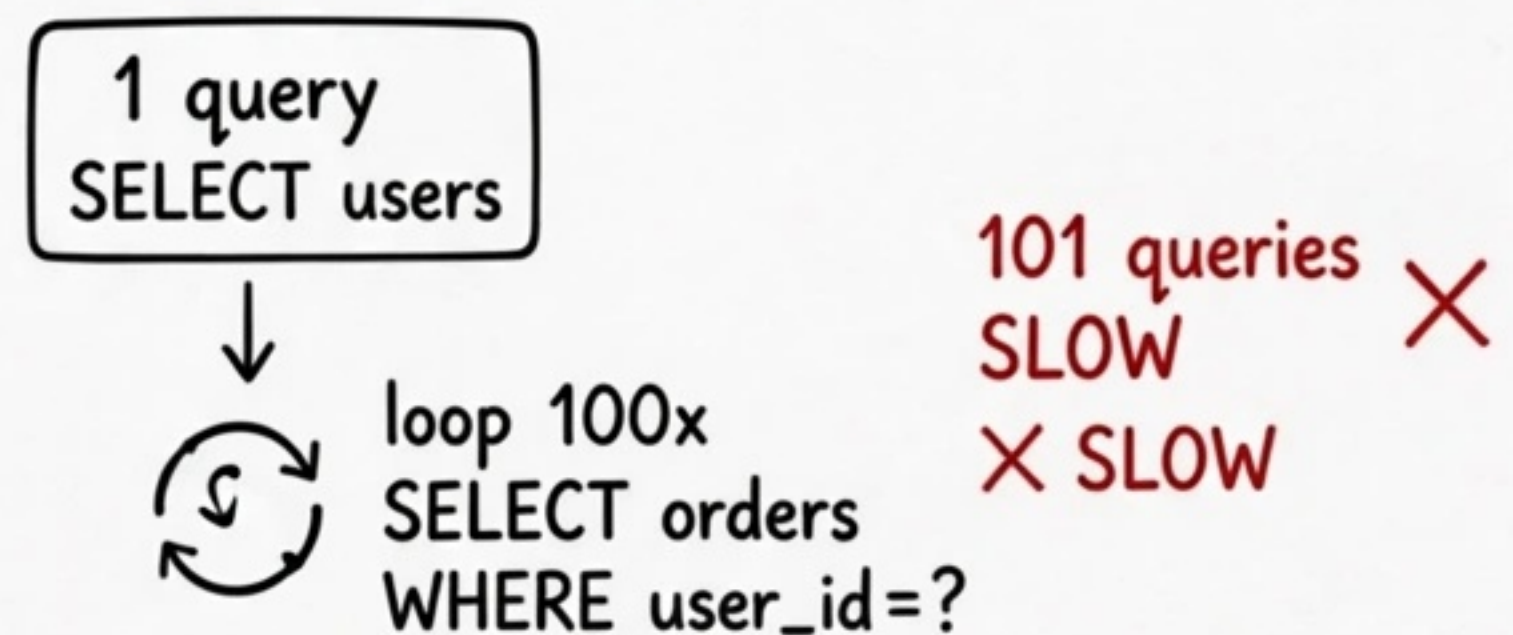
- String, Hash, List, Set, SortedSet (ZSet).

Use cases:

- session store, leadeboard, rate limiting, pub/sub.

`@Cacheable(value="users", key="#id")`

1. N+1 Visual:

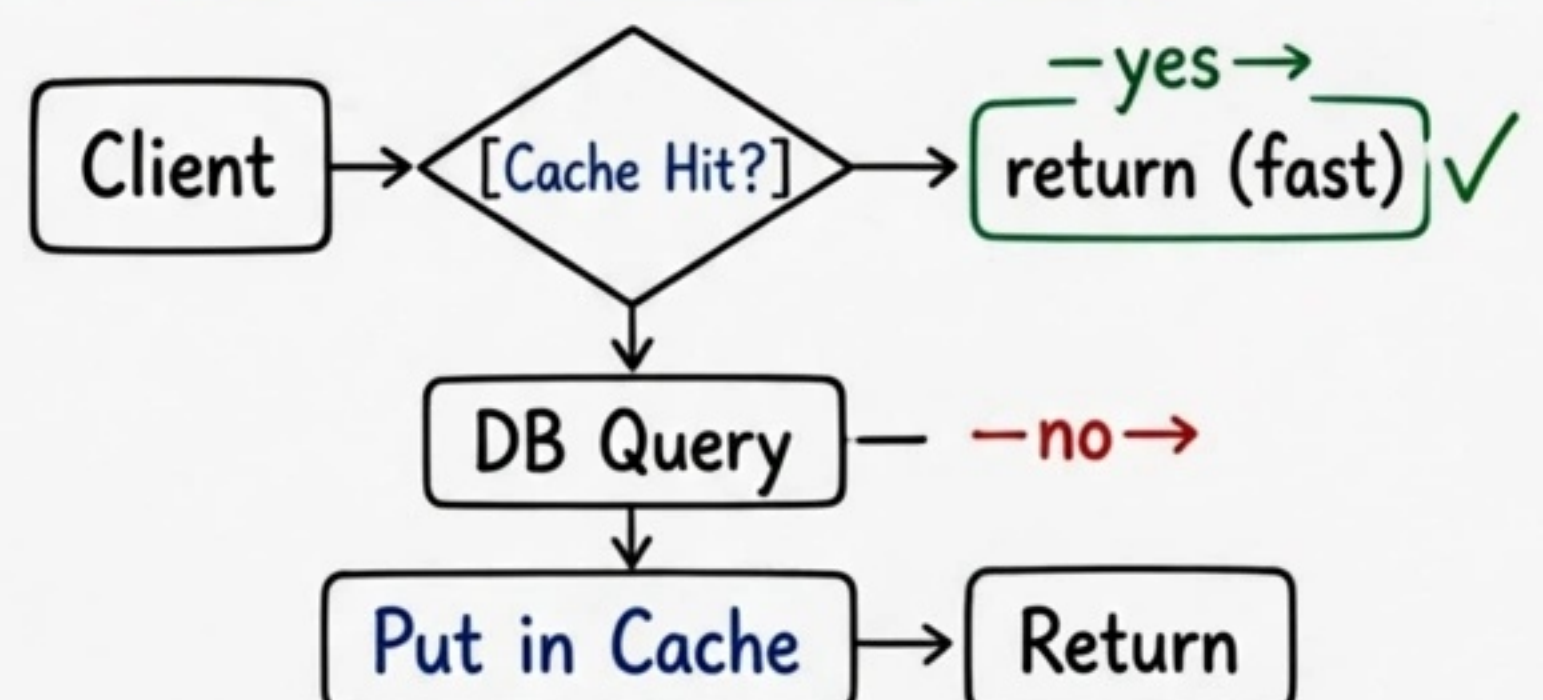


✓ JOIN FETCH → 1 query optimized

`SELECT u JOIN FETCH u.orders`

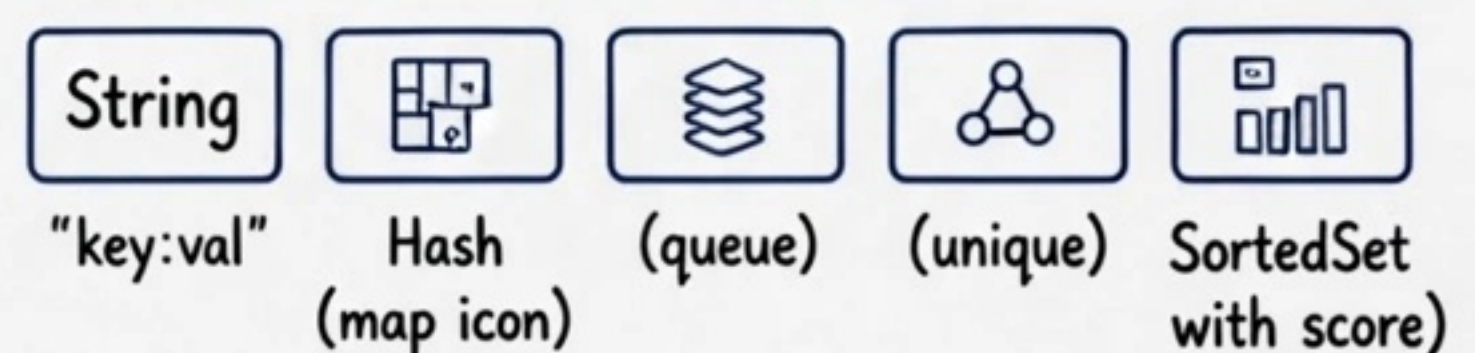
single query → optimized

2. Caching Flow Diagram:



Cache Aside Pattern

3. Redis Types Icons



Part 19

Backend Interview - 50 Questions

From Zero to FAANG

Q35-Q36. Kafka vs RabbitMQ + SQL vs NoSQL?

Q35. Kafka vs RabbitMQ?

Ans: Kafka — log-based • high throughput • distributed streaming • pull model • persistence • ordered by partition • durable • replay.

RabbitMQ — queue-based • low latency • push model • routing via exchange • topic / fanout / direct • delivery.

Kafka for: event sourcing • log analytics • real-time streams.

RabbitMQ for: job queue • RPC • microservice tasks.

Diagram: Flow Kafka vs RabbitMQ

	Kafka	RabbitMQ
Throughput	High	Medium
Latency	Higher	Low
Model	Pull	Push
Persistence	Log / Durable	Queue / Ack
Routing	Partition	Exchange Routing
Ordering	Partition-level	Queue-level

Q36. SQL vs NoSQL?

SQL = RDBMS • structured • fixed schema • ACID • strong consistency • joins • vertical scale • MySQL • Postgres.

NoSQL — schema flexible • BASE • eventual consistency • horizontal scale

Types:

- Document — MongoDB
- Key-Value — Redis
- Column — Cassandra
- Graph — Neo4j

Choose SQL for: transactions • banking.

Choose NoSQL for: big data • social feed • IoT • real-time.

Examples:

Kafka: produce -> topic -> consumer group
pull msgs

SQL: SELECT * FROM users WHERE id = ? ;

SQL vs NoSQL Comparison

	SQL	NoSQL
Structure	Fixed	Flexible
Consistency	Strong	Eventual
Scale	Vertical	Horizontal
Use Cases	Banking	Big Data
Joins	Yes	No


MySQL


Mongo


Redis


Cassandra

Tip: Kafka = streaming & storage,
RabbitMQ = queue & routing.
SQL = ACID transactions,
NoSQL = scale & flexibility.
Choose based on data model!

Swipe Left -> Q37-Q38: Indexing + ACID

Backend Interview - 50 Questions

From Zero to FAANG

Q37-Q38. Indexing + ACID Properties?

Q37. What is Indexing?

- Ans: B-Tree index speeds up search $O(\log n)$ but slows writes.
- Types: single, composite, unique.
- Use EXPLAIN ANALYZE to see index usage.
- Covering index includes all columns needed.
- When NOT to use? low cardinality columns.

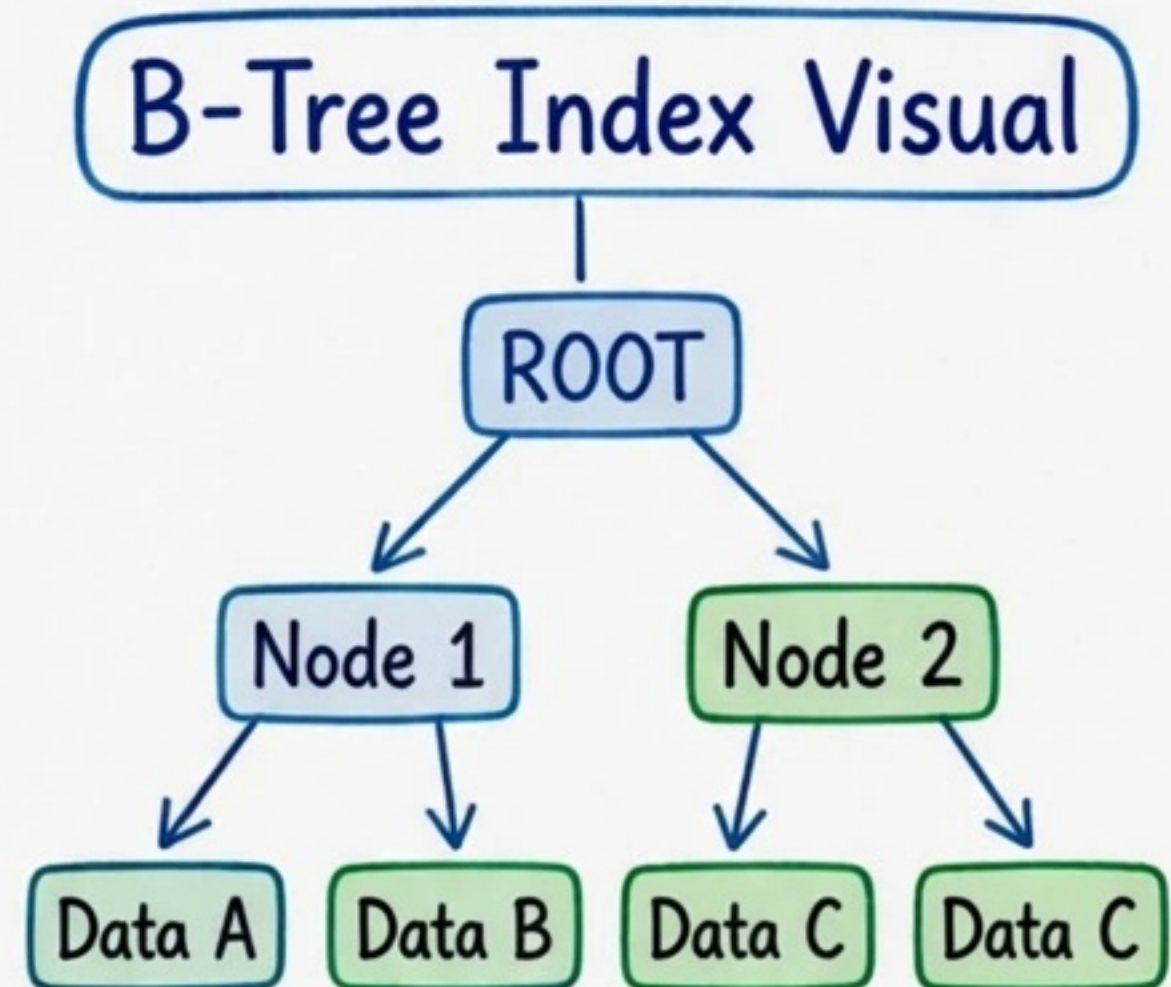


Table Scan → vs **FAST** Index Seek

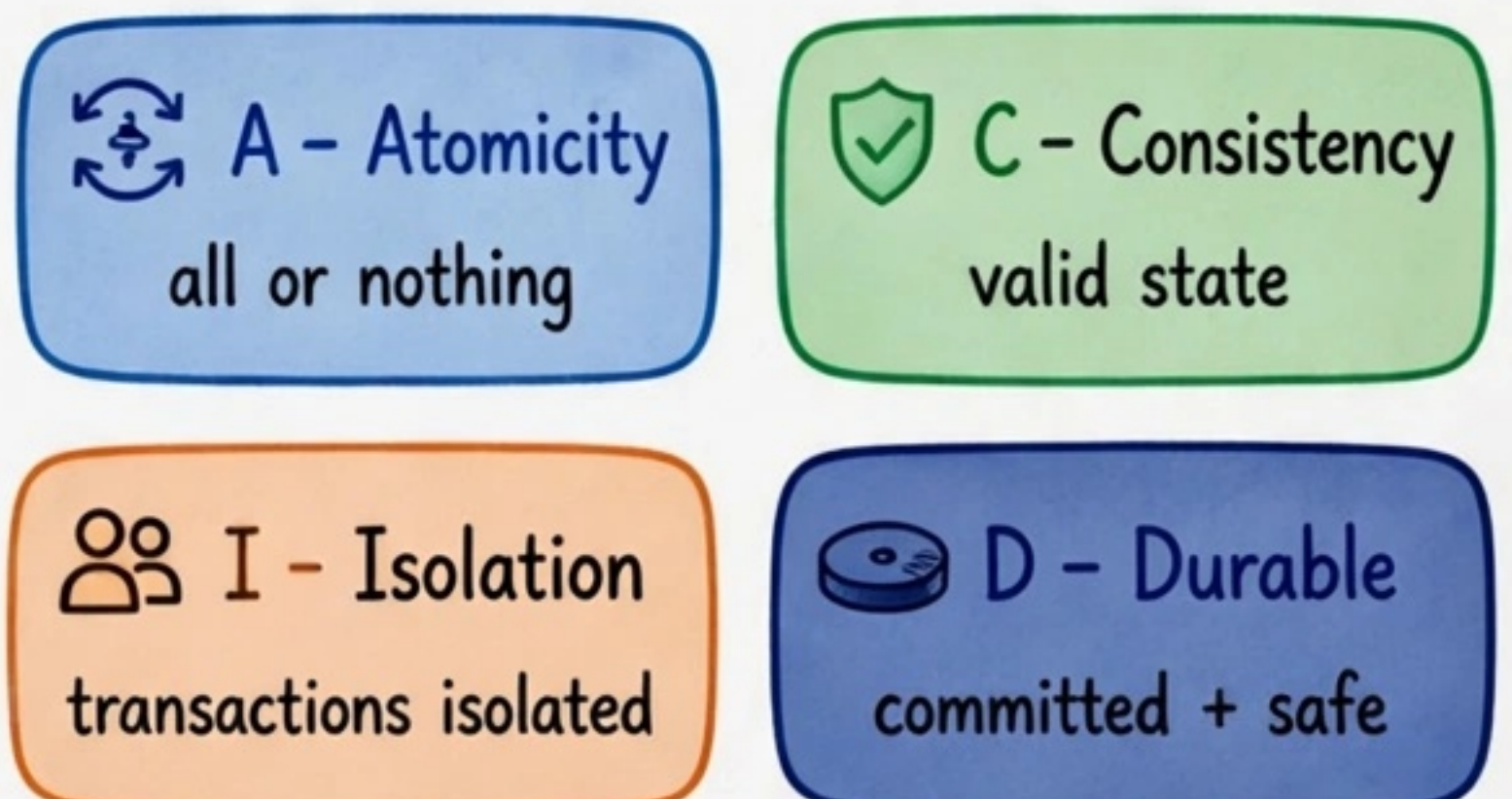
Slow — full rows $O(n)$ → faster

Fast — $O(\log n)$ lookup

Q38. What is ACID?

- Atomicity: all or nothing
- Consistency: valid state → valid state
- Isolation: concurrent transactions isolated
- Durability: committed stays even after crash.
- Implemented via WAL / redo log, locks, MVCC.

ACID Properties



- WAL log | - locks (concurrency) | - MVCC

Swipe Left → Q39-Q40: Isolation Levels + Sharding

Backend Interview - 50 Questions

From Zero to FAANG

Q39-Q40. Isolation Levels + Sharding vs Replication?

Q39. Isolation Levels?

- READ_UNCOMMITTED = dirty read possible, lowest isolation
 - READ_COMMITTED = no dirty read, default in PG, allows non-repeatable + phantom
 - REPEATABLE_READ = no dirty, no non-repeatable, default MySQL, allows phantom
 - SERIALIZABLE = fully isolated, slowest, no anomalies
- Phenomena: dirty read, non-repeatable read, phantom read

Isolation Levels Table

Level	Dirty Read	Non-Repeatable	Phantom
READ_UNCOMMITTED	✓	✓	✓
READ_COMMITTED	✗	✓	✓
REPEATABLE_READ	✗	✗	✓
SERIALIZABLE	✗	✗	✗

lock strength ↑
isolation increases,
anomalies decrease

Q40. Sharding vs Replication?

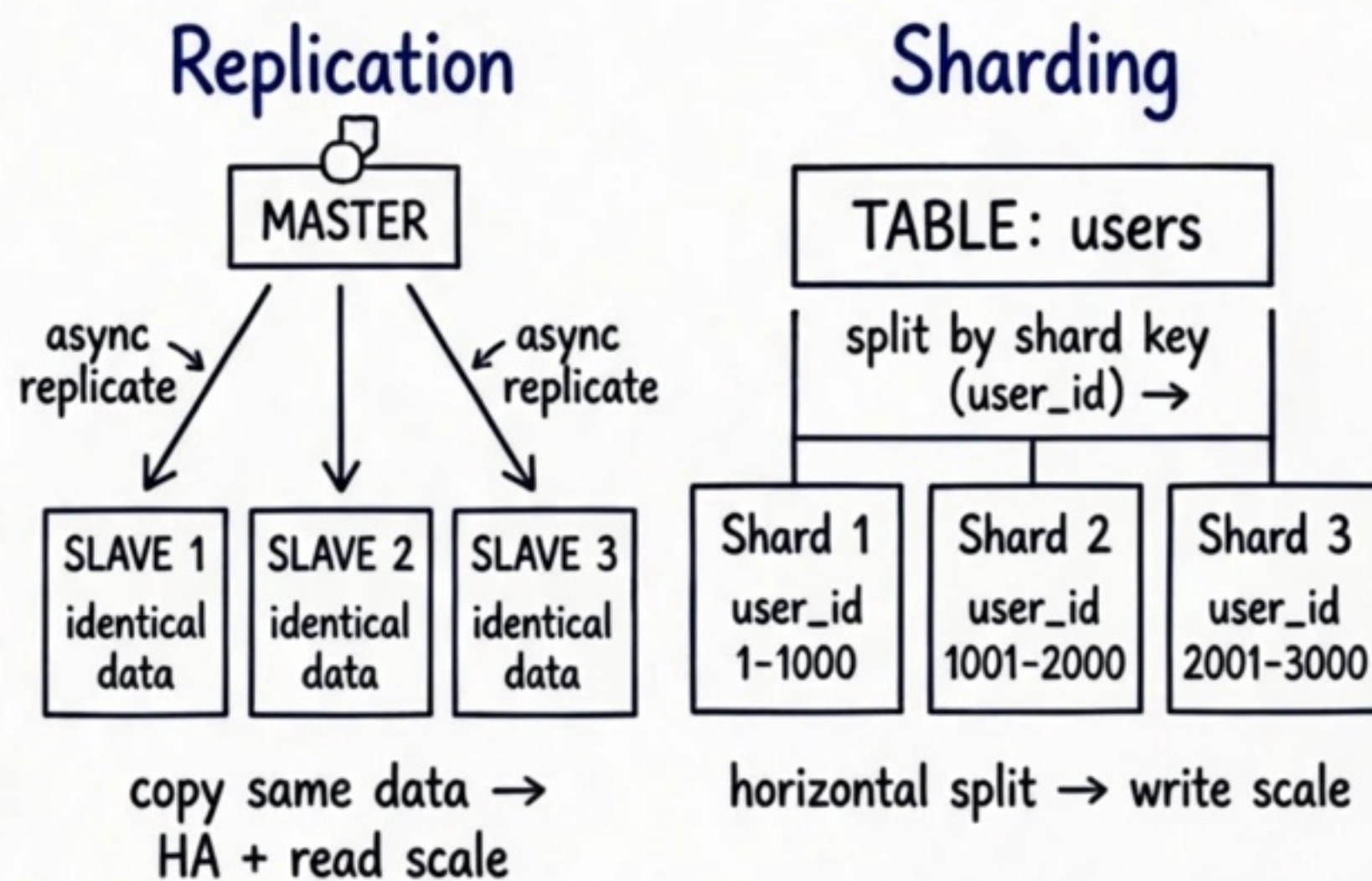
Replication = copy same data to multiple nodes, for HA + read scaling, master → slave

Sharding = split data horizontally across nodes by shard key (user_id), for write scaling + big data

Challenges: resharding, cross-shard joins, choosing shard key

Use both together for production scale

Replication vs Sharding



Swipe Left → Q41-Q42: Load Balancer + Docker

Part 22

@codeera.tech

Backend Interview - 50 Questions From Zero to FAANG

Q41-Q42. Load Balancer L4 vs L7 + Docker?

Q41. Load Balancer?

Ans: Distributes traffic across servers.

- L4 = Transport layer • IP+port • TCP
- L7 = Application layer • HTTP header
- cookie • path based routing • smart
- slow.
- Algorithms: round robin, least connections, IP hash.

Key: L4 is network layer, fast & simple

Q42. Docker?

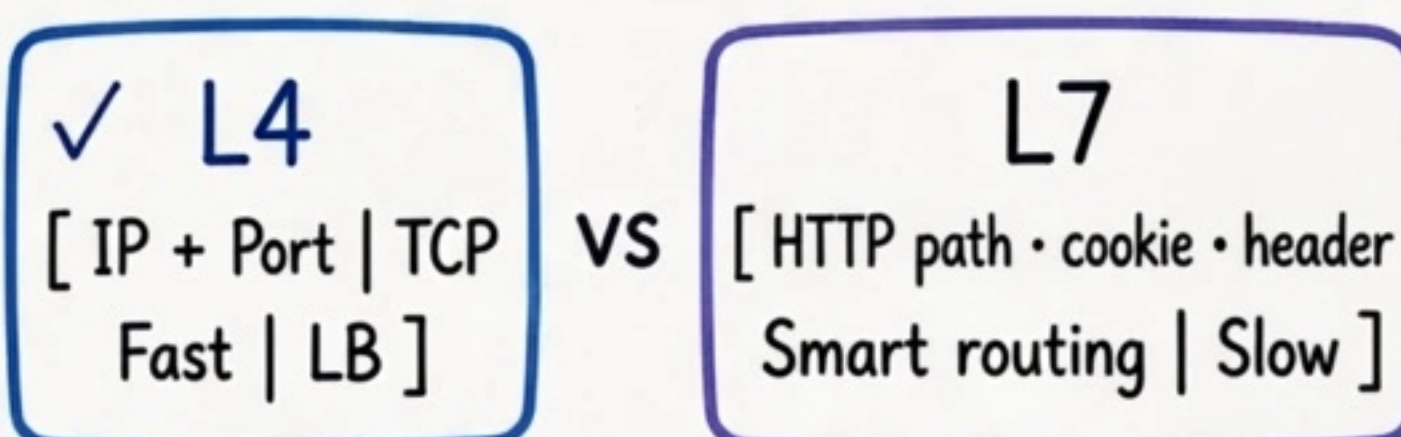
Ans: Containerization vs VM • lightweight.

- Image = ready template • Dockerfile,
- Container = running instance of image,
- Dockerfile = instructions • FROM • RUN • COPY • CMD,
- Registry = Docker Hub stores images,
- Volumes = persistent data.
- Benefits: portability • isolation.

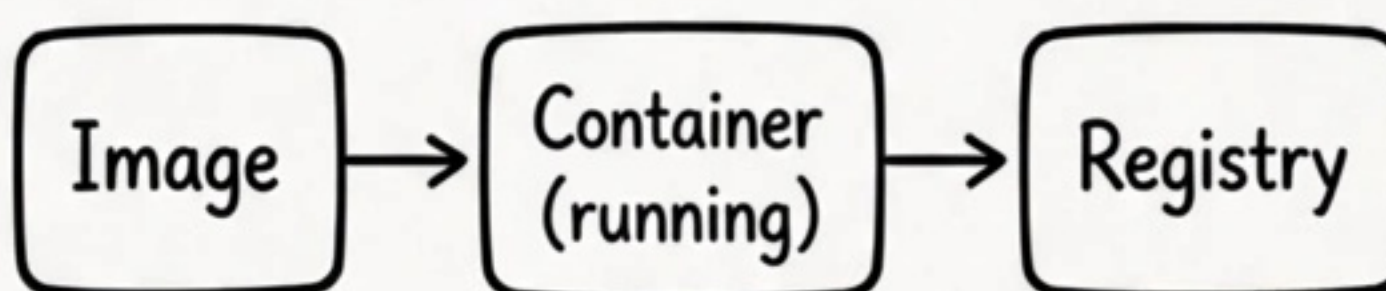
Tip: Docker shares host OS kernel → much lighter than VM

Diagrams:

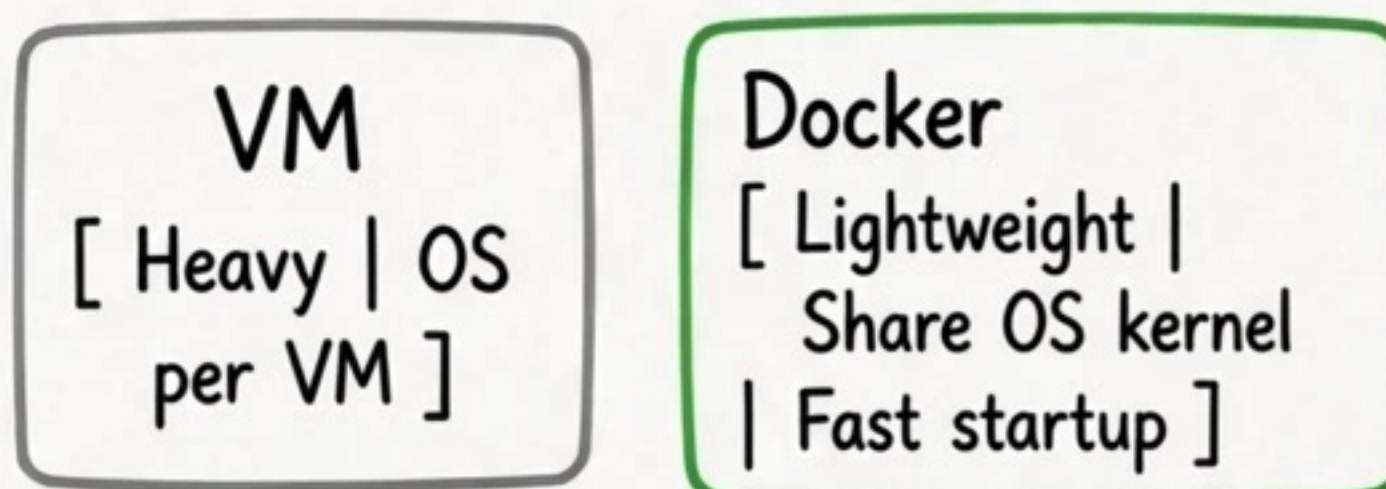
Diagram: L4 vs L7 diagram



Docker Architecture flow:



Comparison box:



Swipe Left → Q43-Q44: K8s + CI/CD

Backend Interview - 50 Questions

From Zero to FAANG

Q43-Q44. Kubernetes + CI/CD Pipeline?

Q43. What is Kubernetes?

Ans: Orchestration platform.

- Pod = smallest deploy unit:
1+ containers
- Deployment manages ReplicaSets
+ rolling update
- Service = stable IP for pods:
ClusterIP | NodePort | LoadBalancer
- ConfigMap, Secret
- Ingress, HPA auto scale
- kubectl commands

Q44. What is CI/CD Pipeline?

- CI = Continuous Integration: build +
test on commit
- Jenkins, GitHub Actions
- Continuous Delivery = deploy to staging
- Continuous Deployment = auto to prod
- Stages: Source → Build → Test →
Docker image → Push Registry →
Deploy K8s

Diagram: K8s Architecture

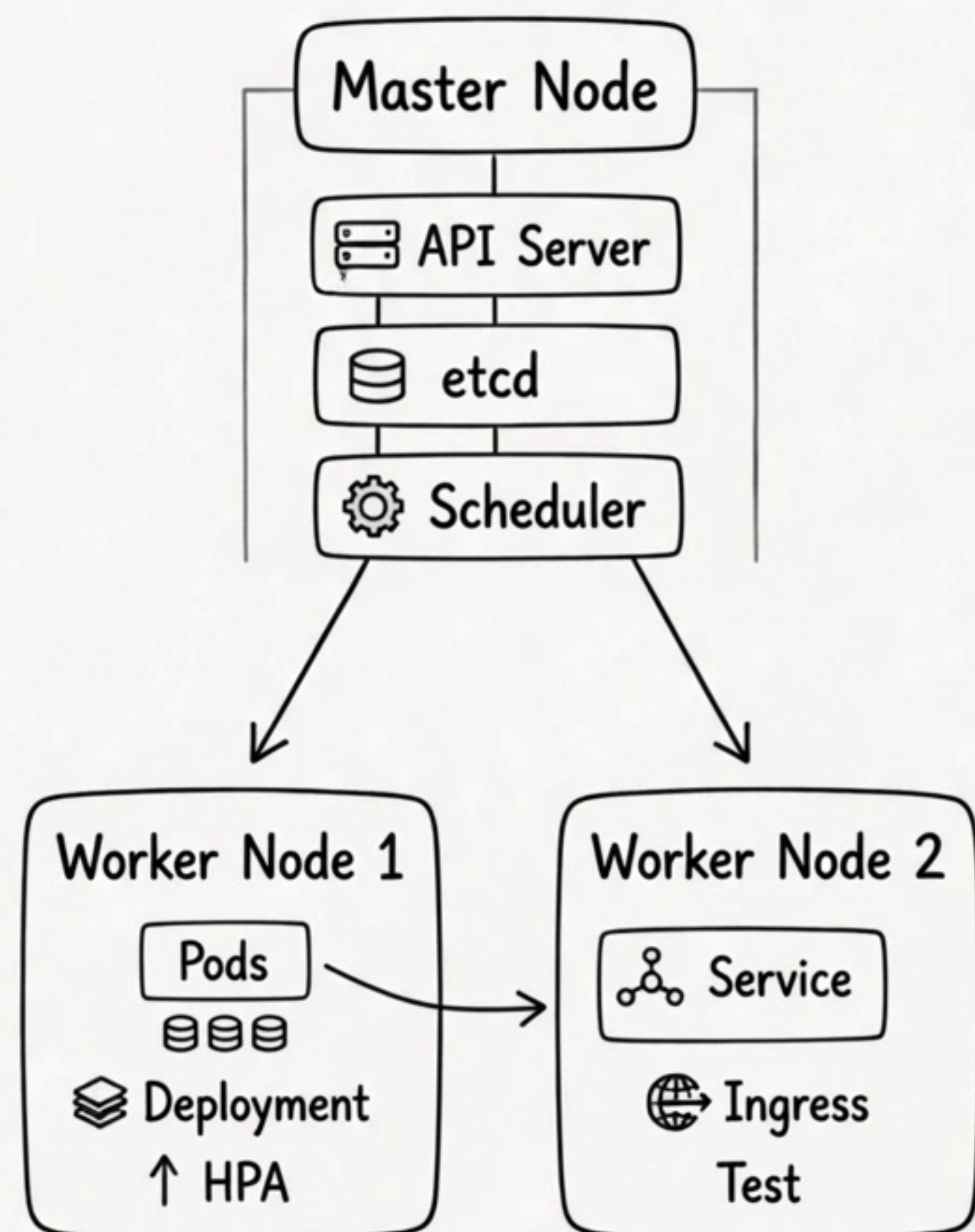
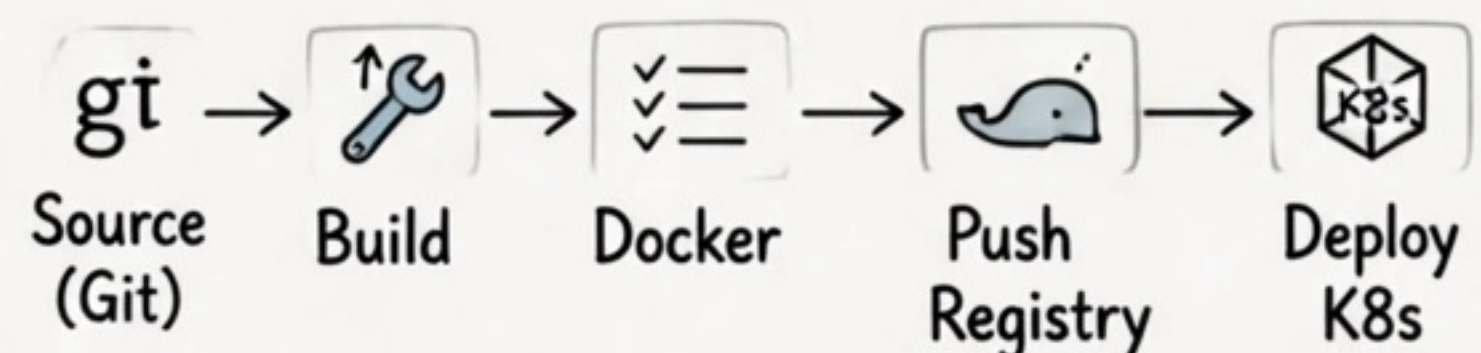


Diagram: CI/CD Flow



Swipe Left → Q45-Q46: System Design URL + Rate Limiter

Part
24

Backend Interview - 50 Questions

From Zero to FAANG

Q45-Q46. System Design URL Shortener + Rate Limiter?

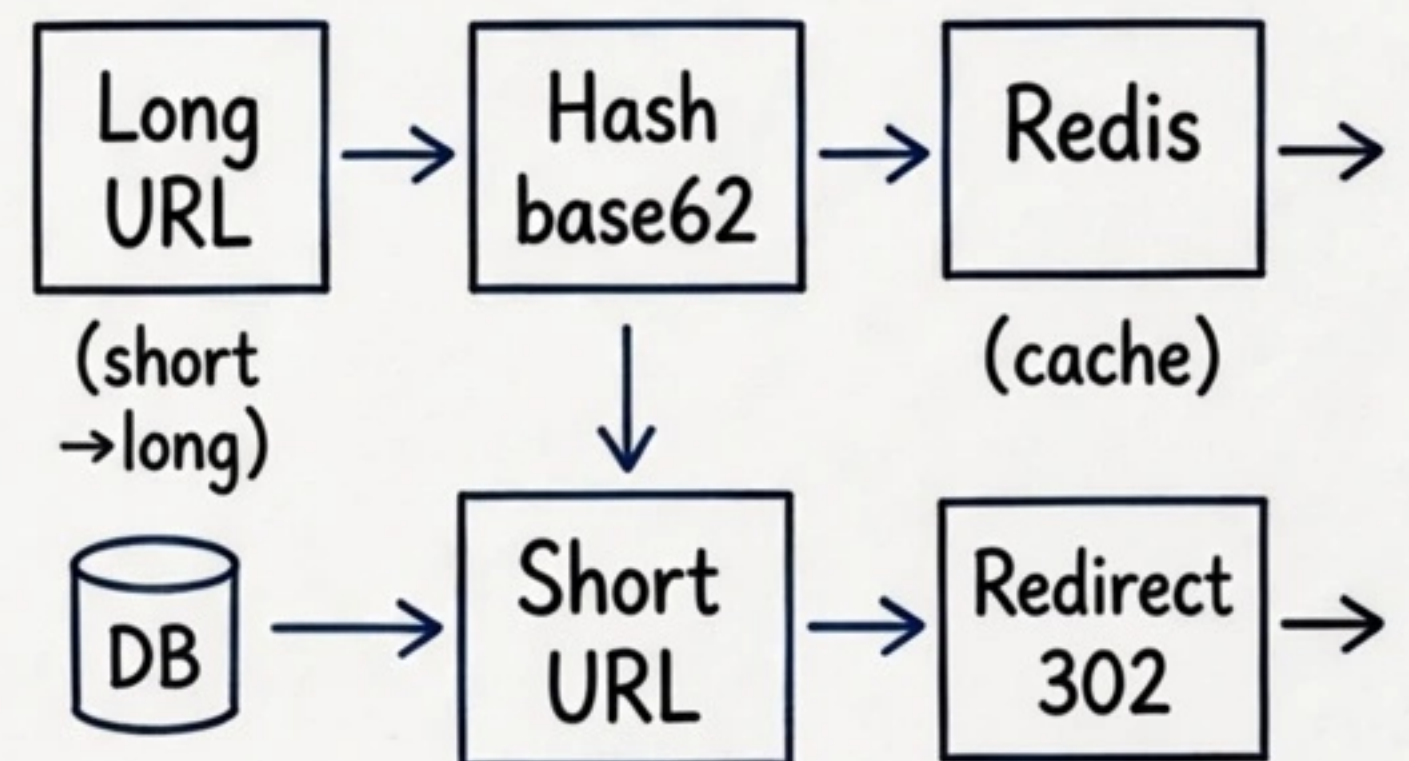
Q45. URL Shortener?

Ans: Requirements: short URL + redirect.

Design: Client POST long URL →
Service generates base62 hash 7 chars
→ Store DB (short→long) + Redis
cache → GET short
→ 302 redirect.

Scale 100M URLs/day. Use Snowflake
for ID. Expiry, analytics.

URL Shortener Flow



Q46. Rate Limiter?

Ans: Limit req per user/time.

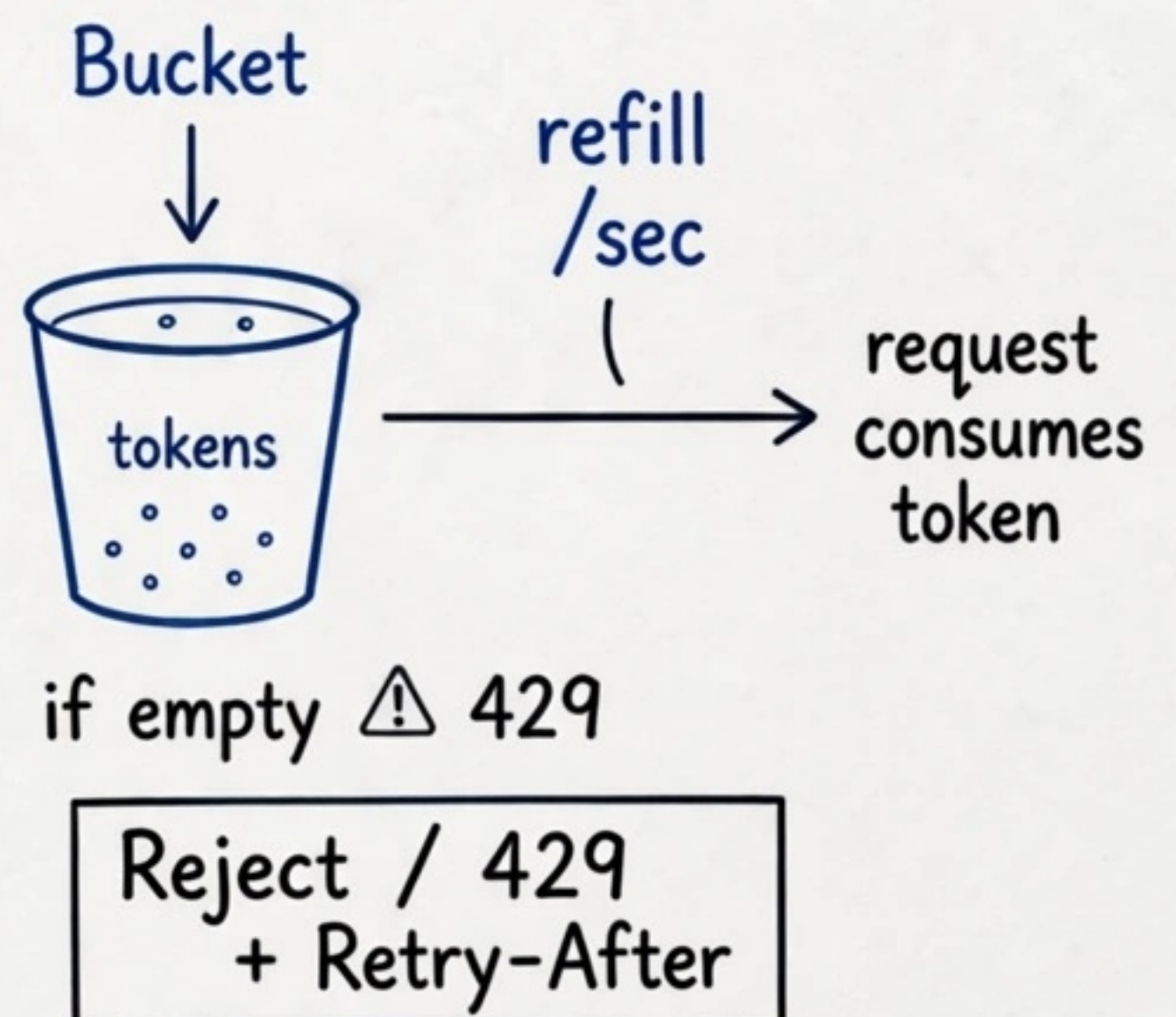
Algos: Token Bucket (tokens refill),
Leaky Bucket (queue leak),

Fixed Window, Sliding Window Log/Counter.

Where? Gateway or middleware
Redis INCR EXPIRE.

Response: 429 Too Many Requests
+ Retry-After.

Rate Limiter Token Bucket



Swipe Left → Q47-Q48: LLD BookMyShow + SOLID

Part 25

Backend Interview - 50 Questions

From Zero to FAANG

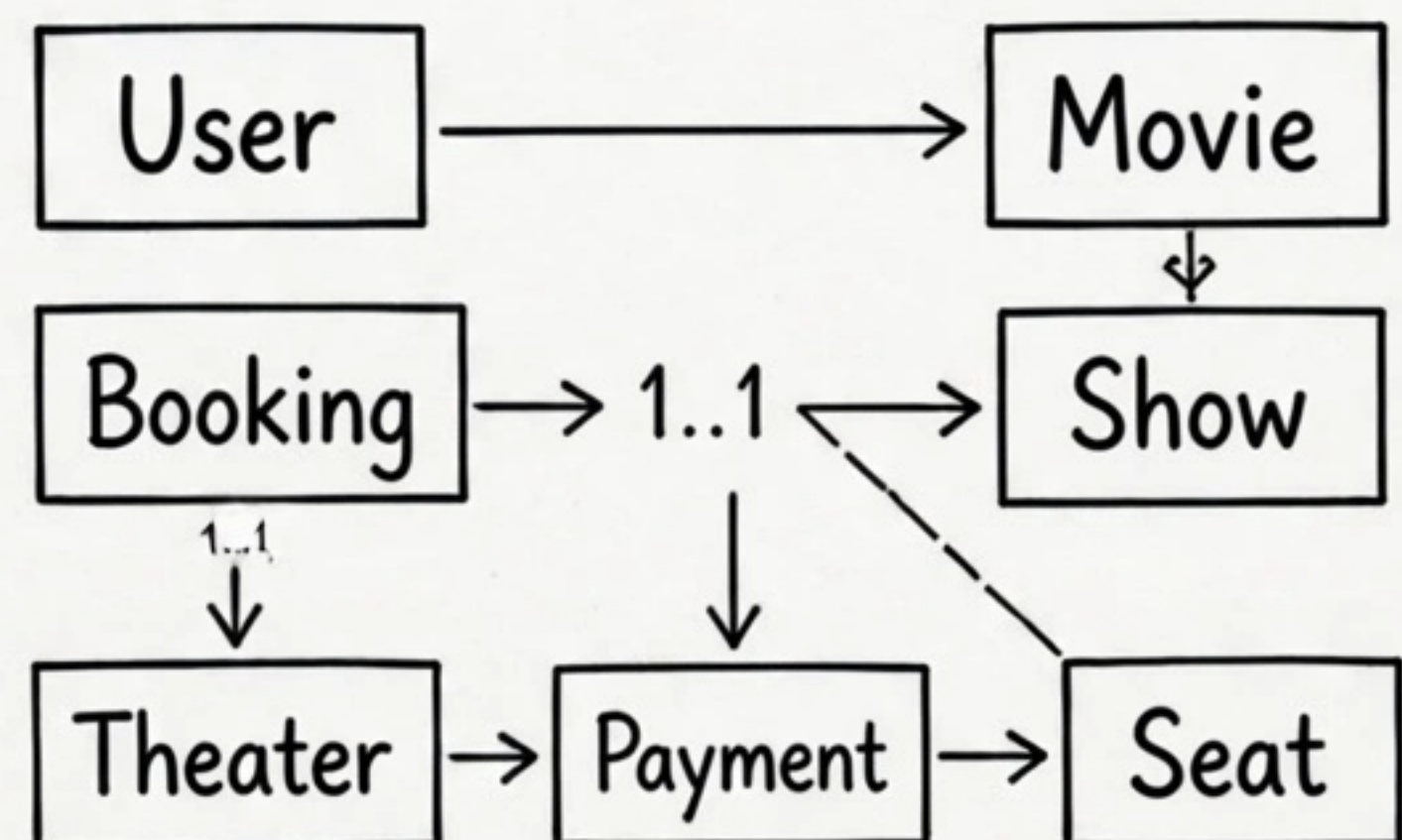
Q47-Q48. LLD BookMyShow + SOLID Principles?

Q47. LLD BookMyShow?

- Classes: User, Movie, Theater, Screen, Seat, Show, Booking, Payment.
- Design patterns: Factory for Payment, Observer for notification, Singleton for config.
- Flow: User searches Movie → Select Theater Show → Select Seats (lock 10min)
- Payment → Booking confirmed.

TIP: Use TTL-lock to prevent double booking

Class Diagram:



SOLID Principles:

S

Single responsibility
• one reason to change

O

Open-closed
• open for extension, closed for modification

L

Liskov
• subtype can replace supertype

I

Interface segregation

Interface segregation
• many small interfaces

Dependency inversion
• depend on abstraction

Part 26

Backend Interview - 50 Questions

From Zero to FAANG

Q49-Q50. OOP for Backend + Mock Interview ? 💡Q49. OOP for Backend ?

4 pillars:

- Encapsulation: private fields, getters
- Abstraction: abstract class, interface
- Inheritance: extends
- Polymorphism: override, overload

Use in backend: Service interface
multiple implementations ↩

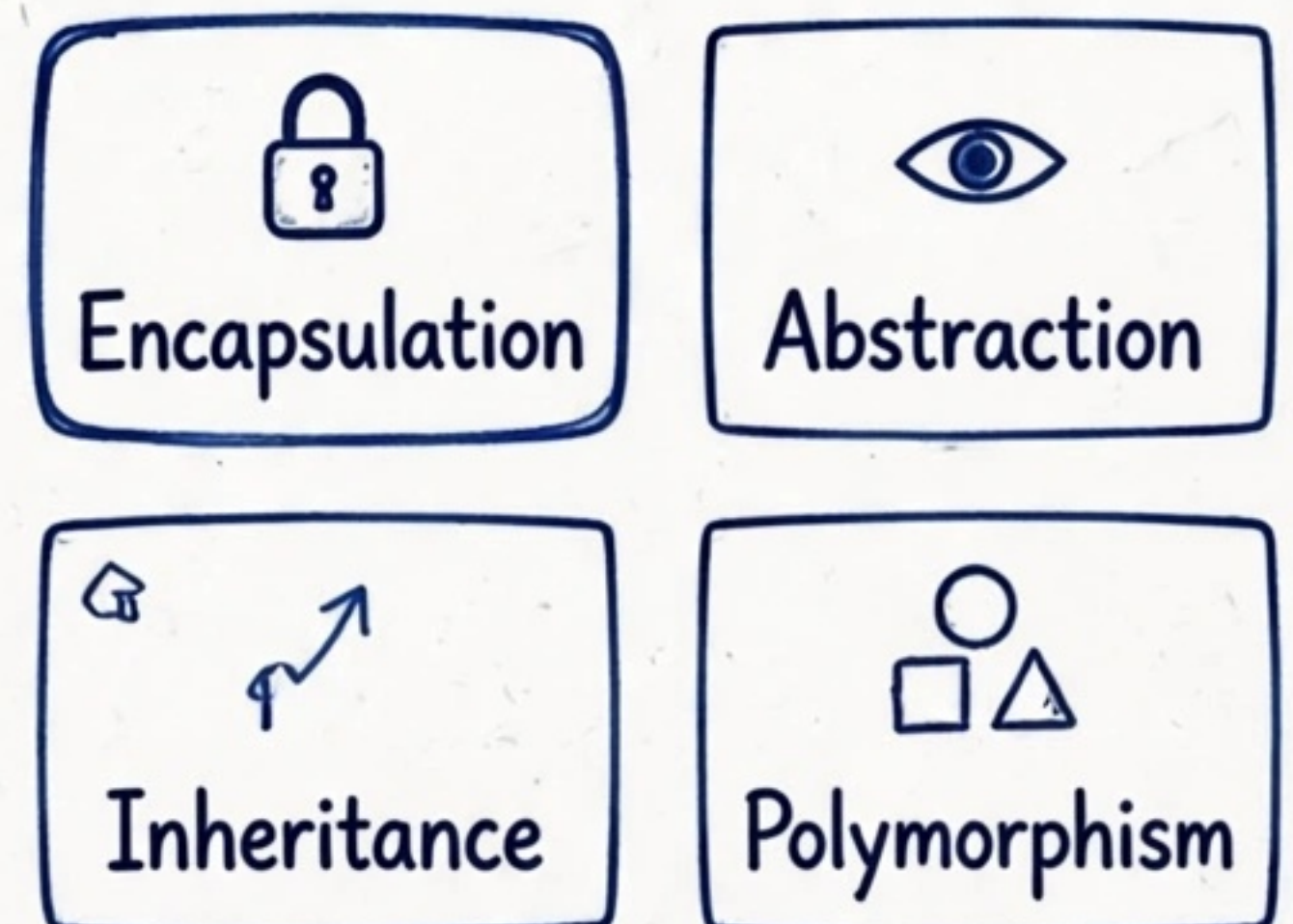
Q50. Mock Interview Final ?

- Tell me about project
- Explain monolith to microservices migration
- Design Instagram backend
- How to handle 1M RPS
- How to prevent N+1
- Saga failure handling
- When to use Kafka vs REST
- CAP in your project
- CI/CD in your company

★ **Golden Tips**

Always start with requirements,
draw architecture, discuss tradeoffs,
mention scalability reliability.

OOP pillars diagram



Mock Interview checklist floychart

